2025

La mise en place d'un cluster kubernetes

Yasmine CHOULADI - Fatine CHAAB - Imane BALOUITI

SMI 2025

Département de Mathématique et Informatique

Projet de Fin d'Etudes

Pour obtenir la Licence en

Sciences Mathématiques et Informatique

---

# *La mise en place d'un cluster kubernetes*

---

Réalisé et soutenu par :

**Yasmine CHOULADI**

**Fatine CHAAB**

**Imane BALOUITI**

Encadré par

***Pr. Noureddine ABGHOUR***

***Faculté des Sciences Aïn Chock***

Membres du Jury

| | | |
|---|---|---|
| ***S. ELNoureddine ABGHOUR*** | ***Fsac*** | ***Encadrant*** |
| | ***Fsac*** | ***Examinateur*** |
| | ***Fsac*** | ***Examinateur*** |

# *Dédicace*

A nos chers parents, pour tous leurs sacrifices, leur amour, leur tendresse, leur soutien et leurs prières tout au long de nos études,

A nos chers collègues, pour leurs encouragements permanents, et leurs soutiens morals,

A nos chers enseignants, pour leurs appuis et leurs encouragements tout au long de nos études,

A nos familles pour leurs soutiens tout au long de notre parcours universitaire,

Que ce travail soit l'accomplissement de vos vœux tant allégués, et le fuit de vos soutiens infaillibles,

Merci d'être toujours là pour nous.

# Remerciements

À l'issue de ce travail, nous souhaitons exprimer nos plus sincères remerciements à l'ensemble du corps enseignant et administratif de la Faculté des Sciences Ain Chock pour la qualité de l'enseignement dispensé et pour l'accompagnement précieux dont nous avons bénéficié tout au long de notre parcours universitaire.

Nous adressons notre profonde gratitude à M. Noureddine ABGHOUR, notre encadrant de projet de fin d'études, pour sa disponibilité, ses conseils éclairés et son soutien constant, qui ont été d'une aide précieuse dans la réalisation de ce travail.

Nos remerciements vont également à tous nos enseignants pour leur engagement et leur dévouement, ainsi qu'à nos camarades pour les échanges constructifs et l'esprit de solidarité qui ont marqué cette formation.

Aucun hommage ne peut être à la hauteur de leurs soutiens permanents, que dieu leurs procure bonne santé et longue vie.

Enfin, nos vifs remerciements aux membres de jury qui nous font l'honneur d'examiner et d'évaluer ce projet.

# *Résumé*

Le présent rapport constitue l'aboutissement des efforts consentis au cours de ces trois années de formation, dans le cadre du projet de fin d'études, en vue de l'obtention de la licence fondamentale en Sciences Mathématiques et Informatiques, délivrée par la Faculté des Sciences Ain Chock.

Ce projet porte sur la conception et le déploiement d'un cluster Kubernetes, une plateforme open-source dédiée à l'orchestration de conteneurs, ayant pour principal objectif l'automatisation du déploiement, de la mise à l'échelle et de la gestion des applications conteneurisées.

Afin de mener à bien cette mission, nous avons procédé à l'installation et à la configuration des nœuds maîtres et travailleurs, tout en intégrant des mécanismes de sécurisation et de supervision du cluster. À travers cette étude, nous avons mis en évidence la flexibilité, la robustesse et l'efficacité de Kubernetes dans des environnements de production.

# *Summary*

This report summarizes the fruit of efforts put in during these three years of study to complete this mission as part of the Graduation Project for a Bachelor's Degree in the Ain Chock Faculty of Science.

The project focuses on the design and deployment of a Kubernetes cluster—an open-source platform dedicated to container orchestration—aiming primarily to automate the deployment, scaling, and management of containerized applications.

To successfully complete this project, we carried out the installation and configuration of both master and worker nodes, while also ensuring the security and monitoring of the cluster. Through this study, we highlight the flexibility, robustness, and efficiency of Kubernetes in real-world production environments.

# Introduction:

Dans un monde en constante évolution, les systèmes d'information doivent s'adapter aux nouvelles exigences en matière de performance, de flexibilité et de réactivité. Aujourd'hui, les entreprises sont de plus en plus nombreuses à adopter des solutions modernes reposant sur des architectures distribuées et des approches orientées microservices, afin de répondre efficacement aux défis du numérique.

Face à cette transformation technologique, la conteneurisation s'est imposée comme une solution incontournable pour simplifier le développement, le déploiement et la gestion des applications. Grâce à des outils tels que Docker, il est désormais possible d'encapsuler une application avec toutes ses dépendances dans un environnement isolé, léger et facilement déployable sur n'importe quelle infrastructure. Toutefois, lorsque le nombre de conteneurs devient important, leur orchestration devient un enjeu majeur. C'est dans ce contexte que Kubernetes intervient comme un outil essentiel, permettant d'automatiser le déploiement, la mise à l'échelle et la supervision des conteneurs à travers un cluster distribué.

C'est dans cette optique que s'inscrit notre projet de fin d'études. Intitulé « **Mise en place d'un cluster Kubernetes** », ce projet a pour objectif de concevoir et de déployer une infrastructure composée d'un nœud maître et de trois nœuds travailleurs, en vue de simuler un environnement de production dans lequel une application simple sera déployée sous forme d'une page web. Ce déploiement, bien que basique, permettra de tester et de valider les différents composants du cluster ainsi que leur bon fonctionnement.

Notre projet est structuré en deux parties principales :

- **Partie I :** Contexte Général du Projet
- Chapitre 1 : Contexte Général du Projet.
- Chapitre 2 : Étude comparative des solutions disponibles et des outils nécessaires à la mise en place d'un cluster Kubernetes.

- Chapitre 3 : Étude des technologies Docker et Kubernetes, avec une mise en évidence de leurs concepts fondamentaux.

- **Partie II :** Conception et Réalisation du Cluster Kubernetes

- Chapitre 1 : Conception de l'architecture du cluster Kubernetes, avec description des composants et des choix techniques.
- Chapitre 2 : Réalisation pratique du cluster, comprenant l'installation, la configuration des nœuds et le déploiement de l'application web.

Finalement, nous clôturant par une conclusion générale.

Ce projet nous a permis d'acquérir une compréhension approfondie des mécanismes liés à l'orchestration des conteneurs, tout en développant des compétences pratiques dans l'administration de systèmes distribués. À travers cette démarche, nous avons posé les bases d'une infrastructure moderne, capable d'évoluer vers des projets plus complexes.

# Table des figures

Figure 1: Logo Docker ..... 17
Figure 2: Logo Podman ..... 19
Figure 3: Logo Containerd ..... 20
Figure 4: Logo Cri-o ..... 21
Figure 5: Logo LXD ..... 23
Figure 6: Logo Kubernetes ..... 25
Figure 7: Logo Docker Swarm ..... 27
Figure 8: Logo Mesos ..... 28
Figure 9: Logo Nomad ..... 29
Figure 10: Logo virtuelbox ..... 31
Figure 11: Logo VMware ..... 33
Figure 12: Logo VMwar Workstation  Player ..... 33
Figure 13: logo Microsoft Hyper-V ..... 34
Figure 14: Logo ubuntu ..... 36
Figure 15: Logo Debian ..... 38
Figure 16: Logo Fedora ..... 39
Figure 17: Logo Kali Linux ..... 40
Figure 18 : Schéma du moteur Docker ..... 43
Figure 19 : Composant du Docker Engine ..... 44
Figure 20: Architecture de Conteneurisation ..... 47
Figure 21: Architecture de Virtualisation ..... 47
Figure 22: Réseau Bridge ..... 49
Figure 23 : Reseaux host ..... 50
Figure 24: Réseau Overlay ..... 50
Figure 25: Réseau Macvlan ..... 51
Figure 26: Architecture Kubernetes ..... 55
Figure 27 : Architecture du cluster ..... 64
Figure 28 : Les machines virtuelles ..... 66

Figure 29: Configuration du Host-Only Networks ..... 67
Figure 30: Commande d'attribution des adresses IP ..... 68
Figure 31 : Commande de vérification de la connectivité ..... 68
Figure 32 : Commande de Mise à jour de la liste des paquets ..... 69
Figure 33 : Installation des paquets essentiels ..... 70
Figure 34 : Commande de mise à jour de la liste ..... 70
Figure 35 : Commande d'ajout la source officielle des paquets Kubernetes ..... 71
Figure 36 : Mise à jour des paquets ..... 72
Figure 37 : les nœuds Master ..... 72
Figure 38 : les nœuds worker ..... 73
Figure 39 : Vérification des versions des composants ..... 73
Figure 40: La page d'accueil Nginx ..... 88

# Table de matières


Dédicace ..... 2

Remerciements ..... 3

Résumé ..... 4

Summary ..... 5

Introduction ..... 6

Partie I: Contexte général du Projet ..... 14

**1. Chapitre 1 : Contexte Général du Projet ..... 15**

1.1. Introduction ..... 15

1.2. Contexte générale du projet ..... 15

1.2.1 Problématique ..... 15

1.2.2 Limites critiques ..... 15

1.3. Solutions proposées ..... 16

1.1. 1.4. Conclusion générale ..... 16

**2. Chapitre 2: Étude comparative des solutions disponibleset outils pour la mise en place d'un cluster Kubernetes ..... 17**

2.1. Guide des technologies de conteneurisation ..... 17

2.1.1 Docker ..... 17

2.1.2. Podman : Une Alternative à Docker ..... 19

2.1.3. Containerd : Le Runtime Minimaliste ..... 20

2.1.4. CRI-O : Runtime Spécialisé pour Kubernetes ..... 21

2.1.5. LXD : Conteneurs Système vs Applicatifs ..... 23

2.1.6. Comparaison des solutions de conteneurisation : ..... 24

2.1.7. Conclusion ..... 25

2.2. Guide des Orchestrateurs de Conteneurs ..... 25

2.2.1. Kubernetes (K8s) : Le Standard Industriel ..... 25

2.2.2. Docker Swarm : La Simplicité Intégrée ..... 27

2.2.3. Apache Mesos : Le Polyvalent ..... 28

2.2.4. Nomad (HashiCorp) : Le Léger et Flexible ..... 29

2.2.5. Comparaison des Orchestrateurs ..... 30
2.2.6. Conclusion ..... 31
2.3. Comparatif des Solutions de Virtualisation ..... 31
2.3.1. Oracle VM VirtualBox - La solution polyvalente et open source ..... 31
2.3.2. VMware - La solution professionnelle ..... 33
2.3.3. Microsoft Hyper-V - La solution Windows native ..... 34
2.3.4. Comparatif détaillé des solutions ..... 35
2.3.5. Guide de choix ..... 35
2.3.6. Conclusion ..... 36
2.4. Comparatif des Distributions Linux majeures ..... 36
2.4.1. Ubuntu - La distribution grand public ..... 36
2.4.2. Debian - La référence de stabilité ..... 38
2.4.3. Fedora - L'innovante ..... 39
2.4.4. Kali Linux - L'outil des pentesters ..... 40
2.4.5. Analyse comparative ..... 41
2.4.6. Conclusion ..... 42
**3. Chapitre 3: Etude de Docker et Kubernetes ..... 43**
3.1. Docker: ..... 43
3.1.1. Introduction ..... 43
3.1.2. La technologie de conteneurs ..... 43
3.1.2.1. Architecture et composants Docker ..... 43
3.1.2.2. Composants du moteur Docker ..... 44
3.1.2.3. Objets Docker ..... 45
3.1.3. Virtualisation VS Conteneurisation ..... 46
3.1.4. Docker networking ..... 48
3.1.5. Quelques commandes de base pour Docker ..... 52
3.1.6. Conclusion ..... 53
3.2. Kubernetes ..... 53
3.2.1. Introduction ..... 53
3.2.2. Définition de Kubernetes ..... 54
3.2.3. Avantages de Kubernetes ..... 54
3.2.4. Architecture de Kubernetes ..... 55
3.2.5. Commandes Kubernetes de Base ..... 57

3.2.6. Conclusion ..... 59
3.3. Conclusion ..... 60
Partie II: Conception et Réalisation du Cluster Kubernetes ..... 61
**1. Chapitre 1 : Conception de l'architecture du Cluster Kubernetes ..... 62**
1.2. Objectif du cluster ..... 62
1.3. Choix du type de cluster ..... 62
1.4. Nombre de nœuds et leur rôle ..... 63
1.5. Architecture réseau ..... 63
1.6. Schema d'architecture ..... 64
**2. Chapitre 2 : Réalisation du cluster kubernetes ..... 65**
**Les étapes de la mise en place d'un cluster kubernetes ..... 65**
2.1. Étape 1 : Installation de VirtualBox et Configuration des Machines Virtuelle ..... 65
2.1.1. Création des Machines Virtuelles ..... 65
2.1.2. Installation du Système d'Exploitation ..... 66
2.1.3. Configuration Réseau dans VirtualBox ..... 66
2.1.4. Vérification de la Connectivité ..... 68
2.2. Étape 2 : Installation et configuration des composants Kubernetes ..... 68
2.2.1. Mise à jour de la liste des paquets ..... 69
2.2.2. Installation des paquets essentiels ..... 69
2.2.3. Importation de la Clé GPG du Dépôt Kubernetes ..... 70
2.2.4. Ajout du Dépôt Officiel de Kubernetes ..... 71
2.2.5. Mise à jour des paquets après ajout du dépôt ..... 71
2.2.6. Configuration des Composants Principaux de Kubernetes ..... 72
2.2.7. Blocage des Mises à Jour de kubelet, kubeadm et kubectl ..... 73
2.3. Étape 3 : Installation de Docker ..... 75
2.3.1. Installation ..... 75
2.3.2. Vérification de l'installation Docker ..... 75
2.3.3. Activation du Démarrage Automatique de Docker ..... 76
3.3.4. Configuration de Docker pour Kubernetes: ..... 76
3.3.5. Désactivation de la Mémoire Swap pour Kubernetes ..... 77
3.3.6. Activation des Modules Overlay et br_netfilter ..... 78
3.3.7. Configuration des Paramètres Réseau pour Kubernetes ..... 78
3.3.8. Application des Paramètres Système ..... 78

2.4. Etape 4 : Configuration du Load Balancer pour le Cluster Kubernetes ..... 79
3.4.1. Installation de Docker ..... 79
3.4.2. Installation de HAProxy pour la haute disponibilité ..... 80
3.4.3. Configuration Frontend et Backend de HAProxy ..... 80
3.4.4. Redémarrage et vérification du service HAProxy ..... 81
2.5. Etape 5: Initialisation des premiers nœuds du cluster ..... 82
2.5.1. **Initialisation du nœud master avec kubeadm** ..... 82
2.5.2. Configuration de kubectl pour Accéder au Cluster ..... 83
2.5.3. Vérification de l'État des Nœuds ..... 84
2.5.4. Installation du réseau Kubernetes ..... 84
2.5.5. Vérification finale des nœuds ..... 85
2.6. Etape 6: Déploiement d'une application simple(Nginx) ..... 85
2.6.1. Création du fichier YAML de déploiement ..... 85
2.6.2. Application du fichier YAML ..... 86
2.6.3. Vérification des pods ..... 87
2.6.4. Vérification du déploiement ..... 87
2.6.5. Vérification des services ..... 87
2.6.6. Vérification de l'Accessibilité de l'Application ..... 88
3.6.7. Simulation d'une Panne de Nœud Worker ..... 89
2.7. Conclusion ..... 90
Conclusion générale ..... 91
Références ..... 91

# Partie I: Contexte général du Projet

La présente partie a pour but de définir l'essentiel du travail d'identification et de la planification du projet, ainsi que l'organisation et le choix méthodologique adopté.

# 1. *Chapitre 1 : Contexte Général du Projet*

## 1.1. Introduction

Kubernetes est aujourd'hui la solution de référence pour l'orchestration de conteneurs. Ce projet consiste à mettre en place un cluster Kubernetes pour répondre aux besoins de sociabilité, de disponibilité et d'automatisation des déploiements, tout en relevant les défis techniques liés à sa mise en œuvre.

## 1.2. Contexte générale du projet

### 1.2.1 Problématique

Avec la croissance rapide des applications cloud-native, les entreprises doivent garantir une haute disponibilité, une scalabilité efficace et des déploiements agiles. Pour répondre à ces enjeux, elles se tournent vers des solutions modernes d'orchestration de conteneurs, parmi lesquelles Kubernetes s'est imposé comme un standard incontournable.

Toutefois, son déploiement, notamment en environnement professionnel, reste complexe. Il nécessite une compréhension approfondie des architectures distribuées, des besoins réseau ainsi que de la gestion fine des ressources. Cette complexité représente un défi de taille pour les équipes techniques qui souhaitent exploiter pleinement le potentiel de Kubernetes.

### 1.2.2 Limites critiques

La mise en œuvre d'un cluster Kubernetes s'accompagne de nombreux défis techniques et organisationnels. Parmi les principales difficultés rencontrées, on peut citer :

- Une courbe d'apprentissage élevée ;
- Une configuration réseau souvent complexe ;
- Des enjeux importants en matière de sécurité ;

- Une gestion fine et parfois difficile des ressources physiques et virtuelles.

Sans une architecture bien conçue et des outils adaptés, la plateforme peut devenir instable, difficile à maintenir et génératrice d'erreurs coûteuses. L'absence de solutions de monitoring et d'automatisation peut également nuire à l'efficacité et à la fiabilité du système.

### 1.3. Solutions proposées

Pour relever ces défis, le projet propose une mise en place progressive d'un cluster Kubernetes en s'appuyant sur des bonnes pratiques éprouvées. L'approche inclut :

- Le choix d'un outil d'installation adapté (kubeadm ou solution managée);
- L'intégration d'outils de surveillance comme Prometheus et Grafana ;
- L'application de politiques de sécurité solides (RBAC, Network Policies) ;
- L'automatisation des déploiements à travers une chaîne CI/CD.

Cette stratégie vise à construire une infrastructure robuste, scalable et facile à gérer, tout en permettant une montée en compétence progressive des membres de l'équipe technique.

### 1.1. 1.4. Conclusion générale

Ce projet de mise en place d'un cluster Kubernetes permet d'appréhender de manière concrète les mécanismes d'orchestration de conteneurs et les avantages qu'ils offrent en matière de gestion des applications. Il constitue une étape essentielle pour toute organisation souhaitant adopter une infrastructure moderne, évolutive et résiliente.

Au-delà des aspects techniques, il permet de mieux comprendre les enjeux liés au DevOps, à la conteneurisation et à la scalabilité des services informatiques.

# 2. *Chapitre 2: Étude comparative des solutions disponibleset outils pour la mise en place d'un cluster Kubernetes*

## 2.1. Guide des technologies de conteneurisation

### 2.1.1 Docker




Figure 1: Logo Docker

Docker est une plateforme open source révolutionnaire qui a transformé la manière dont les applications sont développées, déployées et exécutées. Elle repose sur la technologie de conteneurisation, qui permet d'encapsuler une application et toutes ses dépendances (code, bibliothèques, configurations, binaires) dans un format standardisé appelé image de conteneur. Cette encapsulation garantit que l'application fonctionnera de manière cohérente, quel que soit l'environnement dans lequel elle est exécutée (développement, test, production).

#### 2.1.1.1. Fonctionnalités et Écosystème Docker

Docker simplifie le processus de création et de gestion des conteneurs grâce à :

- ✓ Son interface CLI : (Command Line Interface) Docker propose une interface CLI intuitive et conviviale, permettant aux développeurs et aux administrateurs système de gérer les images, les conteneurs, les volumes et les réseaux à l'aide de commandes simples. Cette interface est un atout majeur pour l'automatisation des tâches et l'intégration dans des scripts de déploiement.
- ✓ Docker Compose : Docker Compose est un outil essentiel pour la gestion d'applications multi-conteneurs. Il permet de définir et de configurer l'ensemble des services nécessaires au bon fonctionnement d'une application (base de données, cache, back-end, front-end, etc.) via un simple fichier YAML. Grâce à cette approche déclarative, le déploiement

d'une stack complète devient rapide, reproductible et facilement maintenable.

- ✓ Docker Swarm : Docker intègre également une solution native d'orchestration appelée Docker Swarm, qui permet de gérer un cluster de machines Docker. Elle offre des fonctionnalités telles que la répartition de charge, la tolérance aux pannes, et la mise à l'échelle automatique des services.

Docker ne se limite pas à la création de conteneurs. Son écosystème inclut aussi :

- Docker Hub : une plateforme centralisée de partage et de distribution d'images, facilitant la réutilisation de composants standards et open-source.
- Dockerfile : un fichier de configuration qui permet de décrire de manière déclarative le processus de création d'une image Docker.
- Volumes et réseaux : des outils intégrés pour la gestion des données persistantes et des communications entre conteneurs.

#### 2.1.1.2. Avantages

- ✓ Portabilité : les conteneurs fonctionnent de la même façon sur toutes les machines (développement, test, production).
- ✓ Isolation : chaque conteneur est isolé des autres, ce qui évite les conflits entre applications
- ✓ Légèreté : plus léger qu'une machine virtuelle, un conteneur consomme moins de ressources.
- ✓ Déploiement rapide : les conteneurs peuvent être créés, supprimés, démarrés ou arrêtés très rapidement.
- ✓ Reproductibilité : permet de recréer le même environnement facilement à l'aide d'un fichier Dockerfile.
- ✓ Réduction des coûts : meilleure utilisation des ressources matérielles

### 2.1.2. Podman : Une Alternative à Docker




Figure 2: Logo Podman

Dans le domaine de la conteneurisation, Docker s'est imposé comme un standard. Cependant, des alternatives émergent, répondant à des besoins spécifiques, notamment en matière de sécurité, d'architecture système et de flexibilité. Podman (Pod Manager) s'inscrit dans cette dynamique en proposant une solution sans démon (daemonless) et orientée vers les environnements Linux modernes.

#### 2.1.2.1. Caractéristiques principales

- ✓ **Architecture sans démon (daemonless)**

Contrairement à Docker, qui repose sur un démon central (dockerd) tournant en arrière-plan, Podman n'utilise aucun service persistant. Chaque commande exécutée avec Podman fonctionne de manière indépendante et directe. Cette absence de démon améliore la sécurité, la stabilité et simplifie la gestion des processus.

- ✓ **Exécution sans privilèges root (rootless containers)**

Podman permet de lancer des conteneurs sans droits administrateur, ce qui limite l'exposition aux vulnérabilités système. Cela répond à une problématique courante de Docker, où les utilisateurs doivent généralement avoir des droits sudo, exposant potentiellement l'ensemble du système. Grâce à cette fonctionnalité, Podman est idéal pour les environnements multi-utilisateurs et pour le respect des bonnes pratiques en cybersécurité.

- ✓ **Compatibilité avec Docker**

Podman est conçu pour être entièrement compatible avec l'écosystème Docker. Il prend en charge les mêmes commandes que Docker (docker run, docker build, etc.), ce qui permet une transition fluide pour les développeurs et

administrateurs déjà familiers avec Docker. De plus, il peut utiliser les mêmes fichiers Dockerfile et interagir avec les images Docker Hub.

- ✓ **Gestion des pods**

Inspiré de Kubernetes, Podman introduit la notion de pod comme unité de base, regroupant plusieurs conteneurs partageant le même réseau et espace de noms. Cela permet de simuler le comportement de Kubernetes à petite échelle, facilitant le développement local avant un déploiement en cluster.

- ✓ **Intégration avec systemd**

Podman offre la possibilité de générer des unités systemd pour gérer les conteneurs comme des services système. Cela est particulièrement utile dans les environnements de production basés sur Linux, où la persistance et le redémarrage automatique sont essentiels.

#### 2.1.2.2. Cas d'usage

- Environnements Linux sécurisés où l'exécution sans privilèges root est une exigence.
- Serveurs de développement ou postes utilisateurs nécessitant un outil de conteneurisation léger, sans dépendance à un démon.
- Déploiement de workloads simples sans l'installation complète de Docker.
- Simulation de comportements Kubernetes via la gestion de pods pour tester localement avant un déploiement à grande échelle

### 2.1.3. Containerd : Le Runtime Minimaliste




Figure 3: Logo Containerd

Containerd est un moteur de conteneurs léger, stable et performant, conçu pour exécuter et gérer le cycle de vie des conteneurs de manière efficace. Il s'agit d'un composant essentiel utilisé en interne par des solutions populaires telles que Docker et Kubernetes, qui s'appuient sur lui pour interagir avec les

conteneurs à un niveau bas.

#### 2.1.3.1. Caractéristiques principales

- **Gestion complète du cycle de vie des conteneurs :** Containerd prend en charge toutes les opérations fondamentales liées aux conteneurs : création, démarrage, arrêt, suppression, gestion des images et des snapshots, etc.
- **Performance et légèreté :** Dépourvu d'interface graphique ou d'outils de haut niveau, Containerd est conçu pour être minimaliste, ce qui en fait un runtime très rapide et peu gourmand en ressources. Il est particulièrement adapté aux environnements de production exigeants.
- **Intégration avec Kubernetes via l'interface CRI (Container Runtime Interface) :** Containerd est compatible avec Kubernetes grâce à son intégration native à l'interface CRI, permettant ainsi à Kubernetes de piloter directement les conteneurs via Containerd, sans passer par Docker.

#### 2.1.3.2. Public cible et cas d'usage

Containerd s'adresse principalement aux équipes DevOps et aux administrateurs système qui recherchent un runtime robuste, fiable et optimisé pour des infrastructures cloud natives complexes. Il est idéal pour les environnements où l'on souhaite un contrôle fin du comportement des conteneurs, sans la surcouche fonctionnelle apportée par Docker.

### 2.1.4. CRI-O : Runtime Spécialisé pour Kubernetes




Figure 4: Logo Cri-o

CRI-O est un moteur de conteneurs spécifiquement conçu pour fonctionner avec Kubernetes. Il a été développé dans l'objectif de fournir une solution légère,

sécurisée et conforme à la norme CRI (Container Runtime Interface), définie par Kubernetes pour standardiser la communication entre l'orchestrateur et le moteur de conteneurs.

#### 2.1.4.1. Caractéristiques principales

- **Conformité totale à la CRI**

CRI-O implémente exclusivement l'interface CRI, ce qui garantit une intégration directe et fluide avec Kubernetes, sans composants superflus. Contrairement à Docker, qui embarque des fonctionnalités additionnelles non requises par Kubernetes, CRI-O se concentre uniquement sur les besoins de l'orchestration.

- **Léger et sécurisé**

En supprimant les éléments non essentiels, CRI-O offre une empreinte système minimale et réduit la surface d'attaque, ce qui le rend particulièrement adapté aux environnements de production où la sécurité est une priorité.

- **Optimisé pour les distributions Kubernetes**

CRI-O est largement utilisé dans des plateformes telles qu'OpenShift, la distribution Kubernetes développée par Red Hat, en raison de sa stabilité, de sa compatibilité avec les normes Linux (comme SELinux, AppArmor) et de sa gestion efficace des ressources.

#### 2.1.4.2. Cas d'usage

CRI-O s'adresse aux architectes cloud et aux équipes DevOps recherchant un runtime spécialisé pour Kubernetes, sans les couches supplémentaires introduites par Docker. Il est idéal pour les clusters Kubernetes en production, où la simplicité, la performance et la sécurité sont des exigences critiques.

### 2.1.5. LXD : Conteneurs Système vs Applicatifs




Figure 5: Logo LXD

LXD est une solution de conteneurisation orientée systèmes d'exploitation complets, qui se distingue des moteurs traditionnels (comme Docker ou Containerd) par sa capacité à gérer des conteneurs système plutôt que des conteneurs applicatifs. Ces conteneurs se comportent comme de véritables machines virtuelles légères, tout en conservant les performances propres à la conteneurisation.

#### 2.1.5.1. Caractéristiques principales

- **Environnement système complet**

Chaque conteneur LXD embarque un système d'exploitation complet, incluant un shell, des services système (comme systemd) et la possibilité d'exécuter plusieurs applications simultanément. L'expérience utilisateur est similaire à celle d'une machine virtuelle, avec une isolation forte tout en bénéficiant des performances natives du noyau hôte.

- **Fonctionnalités avancées**

LXD offre des outils puissants pour la gestion des conteneurs :

- **Snapshots :** création d'instantanés du système pour des sauvegardes ou retours rapides.
- **Sauvegardes et restaurations :** facilité d'exporter et d'importer des conteneurs complets.
- **Migration en direct (live migration) :** déplacement d'un conteneur actif d'un hôte à un autre sans interruption, utile pour la haute disponibilité ou la maintenance.

#### 2.1.5.2. Cas d'usage typiques

LXD est particulièrement adapté aux environnements nécessitant un OS isolé, tels que :

- L'hébergement de serveurs (web, base de données, etc.) dans des conteneurs quasi-équivalents à des VMs.
- La mise en place d'environnements de test ou de développement où chaque conteneur simule une machine indépendante.

Les scénarios de cloud privé, grâce à sa capacité à gérer des dizaines voire des centaines de conteneurs avec une empreinte minimale.

### 2.1.6. Comparaison des solutions de conteneurisation :

Dans un écosystème riche en solutions de conteneurisation, il peut être difficile de choisir l'outil le plus adapté à ses besoins. Le tableau ci-dessous présente une comparaison entre **Docker** et ses principales alternatives, selon plusieurs critères clés.

#### 2.1.6.1. Comparaison des solutions de conteneurisation :

| Critère | Docker | Alternatives |
|---|---|---|
| **Maturité** | Écosystème mature et documenté | Podman/CRI-O moins répandus |
| **Cas d'usage** | Conteneurs applicatifs | LXD pour les conteneurs système |
| **Complexité** | Solution "tout-en-un" | Containerd/CRI-O plus techniques |
| **Sécurité** | Démon centralisé (risque potentiel) | Podman sans démon (+ sécurisé) |

#### 2.1.6.2. Pourquoi Docker reste une référence en conteneurisation

Il constitue le standard industriel de la conteneurisation, reconnu et largement utilisé dans le développement logiciel comme en production.

Il offre une excellente intégration avec les outils DevOps (CI/CD, monitoring, orchestration, etc.).

Sa courbe d'apprentissage est douce, ce qui en fait une solution idéale pour les développeurs, les étudiants ou les équipes débutant avec les

conteneurs.

En somme, Docker reste un choix judicieux pour la majorité des cas d'usage liés aux conteneurs applicatifs, en particulier dans un contexte de développement et de déploiement rapide.

### 2.1.7. Conclusion

Cette partie a permis de dresser un panorama des différentes solutions de conteneurisation, en mettant en avant la polyvalence de Docker grâce à son écosystème mature et sa facilité d'utilisation. Les alternatives telles que Podman, CRI-O, Containerd et LXD apportent des réponses ciblées aux besoins spécifiques liés à la sécurité, à l'orchestration Kubernetes ou à la gestion de conteneurs système.

Le choix de la technologie la mieux adaptée dépendra donc des exigences techniques et des priorités du projet. Cette analyse constitue une base solide pour aborder la conception et la réalisation pratique du cluster Kubernetes, qui sera développée dans la partie suivante.

## 2.2. Guide des Orchestrateurs de Conteneurs

### 2.2.1. Kubernetes (K8s) : Le Standard Industriel




Figure 6: Logo Kubernetes

Kubernetes, souvent abrégé en K8s, est une plateforme open source d'orchestration de conteneurs initialement développée par Google et désormais maintenue par la Cloud Native Computing Foundation (CNCF). Son développement s'appuie sur l'expérience accumulée par Google avec son système interne Borg, ce qui en fait une solution robuste et éprouvée pour la gestion des applications conteneurisées.

#### 2.2.1.1. Fonctionnalités principales :

- **Déploiement automatisé et gestion des réplicas**

Kubernetes permet de définir des déploiements déclaratifs qui assurent la création, la mise à jour et la gestion continue du nombre de réplicas d'une application. Cela garantit une haute disponibilité et une tolérance aux pannes, en redémarrant automatiquement les conteneurs défaillants.

- **Autoscaling dynamique**

Grâce au Horizontal Pod Autoscaler, Kubernetes ajuste automatiquement le nombre de pods en fonction des métriques d'utilisation (CPU, mémoire ou autres métriques personnalisées). Cela optimise l'utilisation des ressources en fonction de la demande réelle, sans intervention manuelle.

- **Load balancing intégré**

Kubernetes intègre des mécanismes de répartition de charge (load balancing) qui distribuent le trafic entrant de façon équilibrée entre les différentes instances d'une application. Cette fonctionnalité améliore la réactivité et la résilience du service.

- **Architecture extensible et modulaire**

Kubernetes est conçu pour être hautement modulaire. Il supporte l'ajout de plugins pour étendre ses fonctionnalités, tels que des plugins réseau (CNI), des systèmes de stockage (CSI), et permet le développement d'opérateurs personnalisés qui automatisent la gestion d'applications spécifiques ou complexes.

- **Gestion avancée du réseau et du stockage**

Kubernetes propose des abstractions réseau permettant aux pods de communiquer entre eux de manière sécurisée et flexible, ainsi qu'une gestion dynamique des volumes de stockage persistants, facilitant le déploiement d'applications stateful.

- **Orchestration multi-cloud et multi-cluster**

Kubernetes supporte le déploiement et la gestion d'applications sur plusieurs environnements cloud ou clusters, facilitant la portabilité et la résilience globale.

#### 2.2.1.2. Atouts majeurs de Kubernetes

- **Adapté aux environnements cloud natifs**

Kubernetes est pensé pour gérer les architectures modernes basées sur des microservices, où l'agilité, la scalabilité et l'automatisation sont cruciales.

- **Gestion efficace des applications complexes à grande échelle**

Sa capacité à orchestrer des milliers de conteneurs simultanément avec des mécanismes de tolérance aux pannes, de mise à jour continue (rolling updates) et de gestion fine des ressources en fait une solution incontournable pour les grandes entreprises et les fournisseurs cloud.

- **Large écosystème et communauté active**

Kubernetes bénéficie d'une communauté dynamique, d'un vaste écosystème d'outils complémentaires et d'une intégration avec la majorité des solutions cloud et DevOps du marché.

### 2.2.2. Docker Swarm : La Simplicité Intégrée

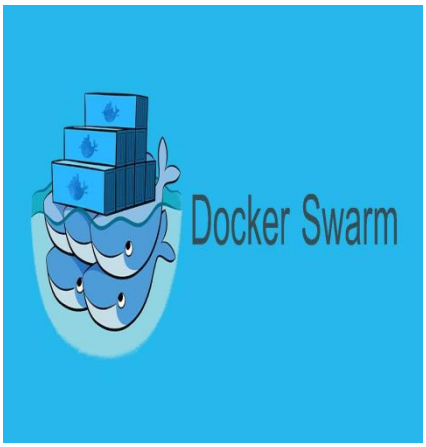



Figure 7: Logo Docker Swarm

Docker Swarm est une solution d'orchestration native développée par Docker, conçue pour offrir une gestion simplifiée des clusters de conteneurs tout en s'intégrant parfaitement à l'écosystème Docker.

#### 2.2.2.1. Fonctionnalités principales :

- **Configuration minimale**

Docker Swarm s'installe et se configure rapidement, avec une intégration native aux outils Docker existants, ce qui facilite sa prise en main et son déploiement.

- **Compatibilité totale**

Il supporte pleinement les fichiers Docker Compose et les commandes Docker CLI, permettant aux utilisateurs de déployer et gérer leurs services avec les mêmes outils qu'ils utilisent pour les conteneurs simples.

- **Courbe d'apprentissage rapide**

Sa simplicité et son interface familière en font une solution adaptée aux petites équipes ou aux projets nécessitant une mise en production rapide sans complexité excessive.

#### 2.2.2.2. Cas d'usage

Docker Swarm est particulièrement recommandé pour le prototypage rapide, les environnements de test, ou les projets où les fonctionnalités avancées d'orchestration ne sont pas indispensables. Il permet ainsi de gérer efficacement des clusters à petite ou moyenne échelle tout en restant accessible.

### 2.2.3. Apache Mesos : Le Polyvalent




Figure 8: Logo Mesos

Apache Mesos se distingue par sa capacité à gérer de manière unifiée des charges de travail très variées, incluant aussi bien des conteneurs que des applications traditionnelles telles que Hadoop, Spark ou des tâches batch.

#### 2.2.3.1. Différenciation majeure

- **Gestion hybride des workloads**

Contrairement à d'autres orchestrateurs centrés uniquement sur les conteneurs, Mesos supporte aussi les charges de travail non conteneurisées, offrant une plateforme unique pour orchestrer des environnements mixtes.

- **Frameworks spécialisés**

Mesos repose sur une architecture modulaire où des frameworks dédiés assurent des fonctions spécifiques :

- Marathon pour l'orchestration et la gestion des applications conteneurisées,
- Chronos pour la gestion des tâches planifiées (jobs batch).

- **Allocation dynamique des ressources**

Le système utilise un modèle d'allocation basé sur des offres et demandes de ressources entre les nœuds du cluster et les frameworks, permettant une utilisation optimale des capacités disponibles.

#### 2.2.3.2. Cas d'usage cible

Apache Mesos est particulièrement adapté aux environnements hybrides où coexistent des applications modernes conteneurisées et des applications héritées (legacy). Il permet ainsi une gestion centralisée et efficace de ces charges hétérogènes dans des infrastructures complexes.

### 2.2.4. Nomad (HashiCorp) : Le Léger et Flexible




Figure 9: Logo Nomad

Nomad, développé par HashiCorp, est une solution d'orchestration légère qui se distingue par sa simplicité d'installation et sa flexibilité d'usage.

#### 2.2.4.1. Avantages clés :

- **Single-binary**

Nomad se présente sous la forme d'un unique binaire, ce qui simplifie grandement son installation, son déploiement et sa maintenance, sans nécessiter de composants additionnels complexes.

- **Support multi-runtime**

Il prend en charge non seulement les conteneurs Docker, mais aussi les applications non conteneurisées, offrant ainsi une grande flexibilité dans la gestion des workloads.

- **Excellente scalabilité**

Conçu pour fonctionner efficacement sur des clusters allant de quelques dizaines à plusieurs milliers de nœuds (jusqu'à 10 000), Nomad répond aux besoins de petites équipes comme de grandes infrastructures.

#### 2.2.4.2. Cas d'usage idéal

Nomad s'adresse aux équipes techniques souhaitant une alternative plus légère et plus simple à Kubernetes, particulièrement adaptée aux environnements hybrides où coexistent des applications conteneurisées et traditionnelles.

### 2.2.5. Comparaison des Orchestrateurs

#### 2.2.5.1. Comparaison des solutions

| Critère | **Kubernetes** | **Docker Swarm** | **Apache Mesos** | **Nomad** |
|---|---|---|---|---|
| Complexité | Élevée | Faible | Modérée | Limité |
| Écosystème | Très riche | Docker-only | Large (frameworks) | Limité |
| Cas d'usage | Cloud natif | Petits clusters | Charges mixtes | Infra hybrides |
| Scalabilité | Excellente | Moyenne | Bonne | Excellente |

#### 2.2.5.2. Pourquoi Kubernetes S'impose ?

**Face à Docker Swarm :**

Kubernetes offre des fonctionnalités avancées telles que le contrôle d'accès basé sur les rôles (RBAC), l'autoscaling automatique des pods, et la possibilité d'utiliser des opérateurs personnalisés pour automatiser la gestion d'applications complexes, des éléments absents ou limités dans Docker Swarm.

**Face à Apache Mesos :**

Kubernetes bénéficie d'une meilleure intégration avec les outils cloud natifs modernes (comme Prometheus, Helm, Istio), facilitant ainsi le déploiement et la gestion d'applications dans des environnements cloud et hybrides.

**Face à Nomad :**

Kubernetes jouit d'une maturité plus avancée et d'une communauté beaucoup plus active, ce qui se traduit par un écosystème riche, un support étendu et une rapidité d'évolution.

### 2.2.6. Conclusion

Kubernetes reste le leader incontesté pour les déploiements multi-cloud, la gestion d'applications microservices complexes et les environnements nécessitant une automatisation poussée. Son écosystème mature, sa flexibilité et son large soutien communautaire en font la solution privilégiée pour des infrastructures à grande échelle et des projets ambitieux.

Cependant, selon les besoins spécifiques, d'autres alternatives peuvent être envisagées.Docker Swarm constitue une option intéressante pour les proofs of concept ou les projets de petite envergure grâce à sa simplicité et sa rapidité de mise en œuvre. Nomad, quant à lui, offre une solution légère et flexible particulièrement adaptée aux petites infrastructures ou aux environnements hybrides. Enfin, Apache Mesos est recommandé pour les environnements complexes où coexistent des workloads conteneurisés et des applications traditionnelles, nécessitant une gestion unifiée et performante.

Le choix de la solution dépendra donc des priorités techniques, de la taille du projet et des exigences opérationnelles.

## 2.3. Comparatif des Solutions de Virtualisation

### 2.3.1. Oracle VM VirtualBox - La solution polyvalente et open source




Figure 10: Logo virtuelbox

Oracle VM VirtualBox est un hyperviseur de type 2, c'est-à-dire un hyperviseur hébergé qui s'installe sur un système d'exploitation hôte existant.

Développé par Oracle, il est à la fois gratuit et open source, ce qui en fait une solution accessible pour de nombreux utilisateurs.

#### 2.3.1.1. Caractéristiques principales

Multiplateforme : Compatible avec les principaux systèmes d'exploitation hôtes, notamment Windows, macOS, Linux et Solaris, ce qui assure une grande flexibilité d'utilisation.

#### 2.3.1.2. Avantages clés

- **Accessibilité**

VirtualBox propose une interface graphique intuitive, adaptée aux débutants, ainsi qu'un support de plusieurs formats de disques virtuels tels que VDI, VMDK ou VHD, facilitant la gestion des machines virtuelles.

- **Polyvalence**

Il supporte un large éventail de systèmes d'exploitation invités, permettant ainsi son utilisation pour le développement, les tests, les formations et d'autres cas d'usage variés.

#### 2.3.1.3. Limitations

En tant qu'hyperviseur de type 2, VirtualBox offre des performances généralement inférieures à celles des hyperviseurs de type 1 (bare-metal), ce qui le rend moins adapté aux environnements de production critiques où la performance et la fiabilité sont primordiales.

### 2.3.2. VMware - La solution professionnelle



Figure 11: Logo VMware



Figure 12: Logo VMwar Workstation  Player

VMware propose une gamme de produits destinés à la virtualisation sur poste de travail, parmi lesquels :

- ✓ VMware Workstation Pro, une solution payante offrant des fonctionnalités avancées,
- ✓ VMware Player, une version gratuite destinée à un usage plus basique.

#### 2.3.2.1. Points forts

- **Performances optimisées**

VMware tire parti de l'accélération matérielle pour garantir une expérience fluide et réactive. Il intègre également des fonctionnalités avancées telles que vMotion (migration à chaud des machines virtuelles) et la gestion des snapshots pour faciliter la sauvegarde et la restauration.

- **Intégration poussée**

La solution est entièrement compatible avec VMware vSphere, permettant une gestion centralisée des machines virtuelles dans des environnements professionnels et d'entreprise.

#### 2.3.2.2. Limitations

Le coût de la version Pro peut représenter un investissement important, généralement compris entre 200 et 300 dollars. Par ailleurs, VMware Workstation est principalement disponible pour les systèmes Windows et Linux, sans support officiel pour macOS.

### 2.3.3. Microsoft Hyper-V - La solution Windows native




Figure 13: logo Microsoft Hyper-V

Microsoft Hyper-V est un hyperviseur de type 1, également appelé hyperviseur bare-metal, ce qui signifie qu'il s'exécute directement sur le matériel physique sans passer par un système d'exploitation hôte intermédiaire.

Cette architecture permet une gestion plus efficace des ressources matérielles et offre des performances supérieures, particulièrement adaptées aux environnements professionnels et serveurs.

#### 2.3.3.1. Caractéristiques principales

Hyper-V est intégré nativement aux systèmes Windows et est disponible sur plusieurs éditions clés :

Windows Server, qui cible les environnements serveurs d'entreprise nécessitant une virtualisation robuste et évolutive,

Windows 10 et Windows 11 Pro/Enterprise, permettant aux utilisateurs avancés et aux professionnels de tirer parti de la virtualisation directement depuis leur poste de travail.

Cette intégration native facilite la gestion centralisée via les outils Windows et assure une compatibilité étroite avec l'ensemble de l'écosystème Microsoft.

#### 2.3.3.2. Avantages

Hyper-V est optimisé pour fonctionner de manière fluide sur les plateformes Microsoft, garantissant des performances fiables et une haute disponibilité. Son intégration avec Active Directory permet une gestion sécurisée des accès et des politiques utilisateurs, tandis que l'interopérabilité avec d'autres services Windows simplifie le déploiement et l'administration des machines virtuelles.

En outre, Hyper-V est inclus gratuitement dans les licences Windows Server et les versions Pro/Enterprise de Windows 10/11, ce qui en fait une

solution économique pour les entreprises déjà investies dans l'écosystème Microsoft.

#### 2.3.3.3. Limitations

Malgré ses nombreux avantages, Hyper-V peut présenter une courbe d'apprentissage assez importante pour les utilisateurs débutants, notamment en raison de la complexité de son interface et de ses fonctionnalités avancées.

L'utilisation de formats de disque propriétaires, tels que VHD et VHDX, peut restreindre la compatibilité avec certains outils ou plateformes tierces, ce qui nécessite parfois des conversions ou adaptations supplémentaires.

Enfin, bien que Hyper-V supporte plusieurs systèmes d'exploitation invités, son support des OS non-Windows est plus limité que d'autres hyperviseurs, ce qui peut poser des difficultés dans des environnements hétérogènes où cohabitent différents types de systèmes.

### 2.3.4. Comparatif détaillé des solutions

| Critère | **VirtualBox** | **VMware Workstation** | **Hyper-V** |
|---|---|---|---|
| Type | Type 2 | Type 2 | Type 1 |
| Licence | Open source | Payant | Gratuit |
| OS hôtes | Multiplateforme | Windows/Linux | Windows uniquement |
| Performances | Moyennes | Élevées | Très élevées |
| Cas d'usage | Développement | Entreprise | Infra Windows |
| Courbe d'apprentissage | Facile | Modérée | Complexe |

### 2.3.5. Guide de choix

#### ➢ Quand choisir VirtualBox ?

- Pour des besoins simples comme les tests, les formations ou le développement.

- Lorsque vous travaillez dans un environnement multi-OS (Linux, macOS, Windows).
- Si votre budget est limité, VirtualBox est une solution entièrement gratuite.

➢ Quand privilégier VMware ?

- Pour des environnements professionnels exigeants où la haute disponibilité et les fonctionnalités avancées sont cruciales.
- Lorsque l'intégration avec l'écosystème VMware, notamment vSphere, est nécessaire.

➢ Quand opter pour Hyper-V ?

- Pour des infrastructures centrées sur Windows et Windows Server.
- Lorsque les performances maximales sont une priorité.
- Si vous bénéficiez déjà d'une licence Windows Pro, Enterprise ou Server.

#### 2.3.6. Conclusion

VirtualBox offre le meilleur compromis entre simplicité et polyvalence pour les petits projets ou les environnements multi-plateformes.

VMware représente une solution premium adaptée aux entreprises recherchant des performances élevées et une intégration avancée.

Quant à Hyper-V, il est parfaitement optimisé pour les environnements Microsoft, offrant des performances maximales dans un cadre Windows natif.

## 2.4. Comparatif des Distributions Linux majeures

### 2.4.1. Ubuntu - La distribution grand public




Figure 14: Logo ubuntu

Ubuntu est une distribution Linux populaire développée par Canonical Ltd., basée sur Debian, et largement reconnue pour sa facilité d'utilisation et sa stabilité. Elle bénéficie d'un cycle de publication régulier, avec des versions LTS (Long Term Support) publiées tous les deux ans, assurant un support à long terme particulièrement adapté aux environnements professionnels.

#### 2.4.1.1. Points forts

Ubuntu se distingue par sa convivialité, notamment grâce à une interface intuitive utilisant GNOME par défaut, rendant l'expérience utilisateur agréable même pour les débutants.

Elle bénéficie d'un support étendu grâce à une documentation riche et une communauté très active, ce qui facilite la résolution des problèmes et l'apprentissage.

Son écosystème est complet, avec des déclinaisons pour les environnements desktop et serveur, ainsi que plusieurs variantes officielles comme Kubuntu, Xubuntu ou Lubuntu adaptées à différents usages et préférences. Le système de paquets Snap permet par ailleurs une gestion simple et moderne des applications.

#### 2.4.1.2. Limitations

Les versions intermédiaires (non-LTS) peuvent parfois manquer de stabilité, ce qui peut poser des risques dans les environnements critiques. Ubuntu est également perçue comme moins personnalisable que certaines distributions plus orientées vers les utilisateurs avancés, comme Arch Linux ou Gentoo.

#### 2.4.1.3. Cas d'usage idéal

Ubuntu est parfaitement adaptée pour :

- Les utilisateurs débutants souhaitant découvrir Linux,
- Les environnements professionnels recherchant une solution stable et supportée,
- Le déploiement de serveurs dans le cloud, notamment via Ubuntu Server, largement utilisé sur des plateformes comme AWS, Azure ou Google Cloud.

### 2.4.2. Debian - La référence de stabilité




Figure 15: Logo Debian

Debian est l'une des plus anciennes distributions GNU/Linux, maintenue par une communauté active depuis 1993. Elle se distingue par son engagement profond envers les principes du logiciel libre, faisant d'elle une référence incontournable dans l'univers Linux.

#### 2.4.2.1. Caractéristiques et Points forts

Debian est une distribution communautaire construite autour d'un processus de développement collaboratif, où la stabilité et la fiabilité priment. Elle est à l'origine de nombreuses distributions dérivées, dont Ubuntu, Mint ou Kali Linux.

Parmi ces principaux points forts :

- **Stabilité extrême :** Debian est réputée pour sa robustesse. Chaque version passe par une phase de test approfondie, ce qui garantit une stabilité maximale, particulièrement adaptée aux environnements critiques. Le cycle de publication, généralement tous les 2 à 3 ans, permet de bénéficier d'un système éprouvé.
- **Grande flexibilité :** Debian propose un accès à plus de 60 000 paquets logiciels, couvrant tous les besoins. Son architecture modulaire et sa compatibilité avec de nombreux environnements en font une base solide pour des systèmes personnalisés ou des distributions dérivées.

#### 2.4.2.2. Défis et public cible

Sa volonté de stabilité conduit parfois à l'inclusion de versions de logiciels plus anciennes, ce qui peut être un frein pour ceux recherchant les dernières nouveautés. De plus, sa courbe d'apprentissage plus raide peut décourager les débutants ou les utilisateurs non familiers avec la ligne de commande.

Debian est particulièrement recommandée pour :

- Les administrateurs systèmes à la recherche de fiabilité,
- Les utilisateurs engagés dans la philosophie du libre,
- Les serveurs critiques nécessitant un fonctionnement sans faille sur le long terme.

### 2.4.3. Fedora - L'innovante




Figure 16: Logo Fedora

Fedora est une distribution Linux sponsorisée par Red Hat, conçue comme un laboratoire d'innovation open source. Elle se caractérise par un cycle de publication rapide (environ tous les 6 mois), intégrant très tôt les technologies émergentes du monde Linux.

#### 2.4.3.1. Particularités et atouts

Fedora sert souvent de terrain d'expérimentation pour les futures fonctionnalités intégrées à Red Hat Enterprise Linux (RHEL). Elle adopte un modèle de développement communautaire, tout en bénéficiant du soutien industriel de Red Hat.

Fedora est souvent à l'avant-garde de l'écosystème Linux. Elle intègre par défaut des solutions avancées telles que SELinux (mécanisme de sécurité renforcée), ainsi que le support natif de Wayland et PipeWire, pour la gestion graphique et multimédia.

Fedora propose des outils adaptés au développement logiciel moderne, comme Podman, une alternative sécurisée à Docker, ou encore Toolbox, qui permet de gérer facilement des conteneurs de développement dans un environnement isolé.

#### 2.4.3.2. Inconvénients

Le rythme rapide des mises à jour rend la maintenance plus exigeante, avec des risques de régressions ou d'instabilités occasionnelles. De plus, la distribution est moins adaptée aux débutants, en raison de la nécessité d'un

certain niveau de maîtrise technique pour tirer pleinement parti de ses fonctionnalités.

#### 2.4.3.3. Utilisation recommandée

Fedora est particulièrement adaptée aux :

- Développeurs souhaitant travailler avec des outils et bibliothèques récents,
- Early adopters désireux de tester les dernières innovations,
- Utilisateurs souhaitant se préparer à l'écosystème RHEL, en environnement professionnel ou serveur.

### 2.4.4. Kali Linux - L'outil des pentesters




Figure 17: Logo Kali Linux

Kali Linux est une distribution Linux spécialisée dans la cybersécurité offensive, développée par Offensive Security. Elle est reconnue mondialement pour sa richesse fonctionnelle dans le domaine du pentesting (tests d'intrusion) et de l'analyse forensique.

#### 2.4.4.1. Spécificités et avantages

Kali Linux intègre plus de 600 outils de sécurité préinstallés, tels que Metasploit, Nmap, Burp Suite, Wireshark, ou encore John the Ripper, couvrant un large éventail d'usages : analyse réseau, exploitation de vulnérabilités, rétro-ingénierie, etc.

Elle peut être utilisée en mode Live (depuis une clé USB ou un CD) sans nécessiter d'installation, ce qui en fait un outil pratique et mobile pour les interventions sur le terrain.

Kali offre une plateforme prête à l'emploi pour les tests d'intrusion, l'analyse forensique, et plus généralement l'audit de sécurité.

Grâce au mode Live, les utilisateurs peuvent lancer Kali sur n'importe quelle machine sans compromettre le système hôte.

#### 2.4.4.2. Mises en garde

**Risque d'usage inapproprié :** L'accès à des outils puissants peut entraîner une utilisation illégale s'ils sont employés sans autorisation. Il est donc essentiel de respecter les cadres légaux en vigueur.

**Non adapté à un usage quotidien :** Kali n'est pas conçu pour une utilisation bureautique classique ; il manque d'optimisations pour la productivité quotidienne ou la gestion multimédia.

#### 2.4.4.3. Public cible

Kali Linux est particulièrement destiné à :

- Les professionnels de la sécurité informatique,
- Les étudiants en cybersécurité,
- Les formateurs et centres spécialisés dans la sécurité offensive.

### 2.4.5. Analyse comparative

| Critère | **Ubuntu** | **Debian** | **Fedora** | **Kali Linux** |
|---|---|---|---|---|
| Public | Grand public | Experts | Technophiles | Spécialistes |
| Stabilité | Élevée (LTS) | Très élevée | Moyenne | Variable |
| Innovation | Progressive | Conservatrice | Avant-garde | Ciblée |
| Support | Commercial | Communautaire | Hybrid | Niche |
| Cas typique | Poste de travail | Serveurs | Dev/Testing | Pentesting |

#### 2.4.5.1. Avantages distinctifs d'Ubuntu

Ubuntu est conçu pour offrir une prise en main simple et intuitive. L'installation est facilitée par un installateur graphique moderne, et la prise en charge des pilotes propriétaires (Wi-Fi, GPU…) est intégrée, ce qui garantit une compatibilité matérielle étendue dès la première utilisation.

La distribution propose plusieurs éditions adaptées à différents contextes :

- Ubuntu Desktop pour les utilisateurs classiques,
- Ubuntu Server pour les déploiements back-end,
- Ubuntu Cloud/Pro pour les environnements virtualisés et conteneurisés.

Elle supporte également les paquets Snap, qui offrent une installation universelle et isolée des applications.

Canonical, l'éditeur d'Ubuntu, propose des offres de support technique payant, incluant des services de sécurité, de maintenance et de conformité, ce qui en fait une solution fiable pour les entreprises.

### 2.4.6. Conclusion

Le choix d'une distribution Linux dépend avant tout du profil de l'utilisateur, des besoins spécifiques du projet et de l'environnement cible. Chacune des distributions présentées possède des atouts distinctifs qui répondent à des cas d'usage bien définis.

Ubuntu se distingue par sa convivialité, sa flexibilité et son fort écosystème, ce qui en fait un excellent choix pour les débutants comme pour les professionnels.

Debian, réputée pour sa stabilité et sa rigueur, s'adresse davantage aux environnements critiques et aux administrateurs systèmes.

Fedora incarne l'innovation, en intégrant rapidement les technologies émergentes, ce qui en fait une plateforme idéale pour les développeurs et les early adopters.

Kali Linux, quant à elle, reste incontournable pour les experts en cybersécurité, en offrant une boîte à outils complète pour les tests d'intrusion et l'analyse forensique.

En somme, aucune distribution n'est « meilleure » en absolu : le choix optimal dépend de l'usage attendu, du niveau d'expertise et des contraintes techniques du projet. Ce panorama permet ainsi de mieux s'orienter dans l'écosystème Linux, en tenant compte à la fois de la facilité d'utilisation, des performances, du support et de la spécialisation.

# 3. Chapitre 3: Etude de Docker et Kubernetes

## 3.1. Docker:

### 3.1.1. Introduction

Un conteneur, c'est une sorte de boîte dans laquelle on met une application avec tout ce qu'il lui faut pour fonctionner (bibliothèques, configurations, etc.). Ça permet d'exécuter l'application partout sans souci, rapidement et sans dépendre de la machine. Contrairement aux machines virtuelles, les conteneurs sont plus légers, rapides et faciles à gérer.

### 3.1.2. La technologie de conteneurs

Docker est une plateforme qui permet de créer, lancer et gérer ces conteneurs. L'idée, c'est que peu importe où on déploie l'application (en développement, test ou production), elle fonctionnera toujours pareil.

#### 3.1.2.1. Architecture et composants Docker

Docker utilise une architecture client-serveur. Le client Docker communique avec le démon Docker, qui sert à créer, exécuter et distribuer les conteneurs Docker. Le client et le démon Docker communiquent à l'aide d'une API REST, via des sockets UNIX ou une interface réseau.

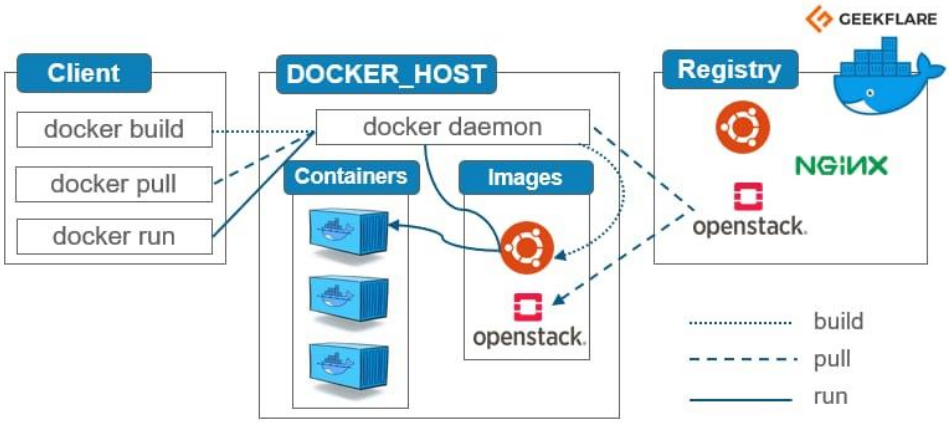



Figure 18 : Schéma du moteur Docker

**L'architecture de docker se compose de cinq composants principaux:**

**a) Docker Daemon :** écoute les requêtes de l'API Docker et gère les objets Docker

tels que les images, les conteneurs, les réseaux et les volumes.

**b) Clients Docker** : avec l'aide des clients Docker, les utilisateurs peuvent interagir avec Docker. Le client Docker fournit une interface de ligne de commande (CLI) qui permet aux utilisateurs d'exécuter et d'arrêter des commandes d'application vers un démon Docker.

**c) Docker Host :** fournit un environnement complet pour exécuter et exécuter des applications. Il comprend le démon Docker, les images, les conteneurs, les réseaux et le stockage.

**d) Docker Registry:** stocker les images Docker. Docker Hub est un registre public que tout le monde peut utiliser, et Docker est configuré pour utiliser des images sur Docker Hub par défaut. Vous pouvez y exécuter votre propre registre.

**e) Les images Docker :** sont des modèles en lecture seule que vous créez à partir d'un ensemble d'instructions écrites dans Dockerfile. Les images définissent à la fois ce que vous voulez que votre application packagée et ses dépendances ressemblent aux processus à exécuter lors de son lancement.

### 3.1.2.2. Composants du moteur Docker




Figure 19 : Composant du Docker Engine

**Les principaux composants du moteur Docker :**

**a) Docker Daemon :** Gère les images Docker, les conteneurs, les réseaux et volumes. Il écoute également les requêtes de l'API Docker et les traites.

**b) API REST de Docker Engine :** Une API développée par Docker qui interagit avec le démon.

**c) Docker CLI :** L'interface de ligne de commande pour communiquer avec le démon Docker. Avec le moteur Docker, vous pouvez exécuter des applications conteneurisées sur n'importe quelle infrastructure. Cette configuration contribue au moteur d'exécution de conteneurs de Docker, leader de l'industrie.

### 3.1.2.3. Objets Docker

Lorsque nous travaillons avec Docker, nous utilisons des images, des conteneurs, des volumes, des fichiers, des réseaux ; ce sont tous des objets Docker.

**a) Image Docker**

Une image Docker, c'est un paquet qui contient tout ce qu'il faut pour lancer une application : le code, les dépendances, les outils. C'est un modèle en lecture seule qu'on utilise pour créer des conteneurs.

**b) Conteneur Docker**

Un conteneur, c'est un espace isolé où une application peut s'exécuter sans déranger le reste du système. Il contient tout ce dont l'appli a besoin, ce qui le rend facile à déplacer d'un environnement à un autre. Avantages des conteneurs Docker : Standard : Docker a simplifié l'usage des conteneurs, aujourd'hui c'est devenu la norme. Léger : Ils partagent le noyau du système, donc ils utilisent moins de ressources que des machines virtuelles. Sécurisé : Ils isolent bien les applis, ce qui renforce la sécurité. Les Conteneurs sont plus rapides, plus efficaces et plus pratiques que les VM.

**c) Volumes**

Les volumes servent à stocker les données des conteneurs de manière

persistante. Contrairement aux données internes du conteneur, les volumes restent même si le conteneur est supprimé. Et en plus, ils n'augmentent pas la taille du conteneur.

#### d) Dockerfile

Un Dockerfile, c'est un petit script qui décrit étape par étape comment construire une image Docker. Il sert à automatiser la création des images.

#### e) Docker Compose

Docker Compose permet de lancer plusieurs conteneurs en même temps, comme un seul service. Par exemple, si une application a besoin de NGINX et MySQL, Docker Compose peut tout démarrer ensemble avec un seul fichier.

### 3.1.3. Virtualisation VS Conteneurisation

#### 3.1.3.1. Machines virtuelles

Les machines virtuelles (VM) sont des systèmes complets simulés grâce à un hyperviseur, qui permet de créer une "copie virtuelle" du matériel : processeur, mémoire, disque dur, etc. Chaque VM fonctionne avec son propre système d'exploitation, ce qui donne une grande isolation, mais demande aussi beaucoup de ressources. Comme elles embarquent un OS entier, les VMs sont souvent lourdes (plusieurs gigas) et peuvent ralentir les performances globales du système. Elles restent utiles dans certains cas, mais sont de plus en plus remplacées par les conteneurs pour des besoins plus légers et flexibles.

#### 3.1.3.2. Conteneurisation

La conteneurisation, c'est une méthode qui permet de faire tourner une application dans un environnement isolé appelé conteneur. Contrairement à la virtualisation, ici ce n'est pas tout le système qui est simulé, seulement l'environnement nécessaire pour l'exécution (comme le système de fichiers, le réseau, la mémoire).Tout ce dont l'application a besoin (fichiers, bibliothèques, dépendances) est inclus dans ce conteneur. Celui-ci utilise ensuite directement le noyau (kernel) du système d'exploitation de la machine hôte, qui gère les ressources et la communication entre les composants.

Ce système permet de déplacer facilement une application d'un

environnement à un autre (à condition qu'ils aient le même OS), tout en consommant moins de ressources (RAM, CPU, etc.) qu'une machine virtuelle.

### 3.1.3.3. La différence entre Conteneurisation et Virtualisation :

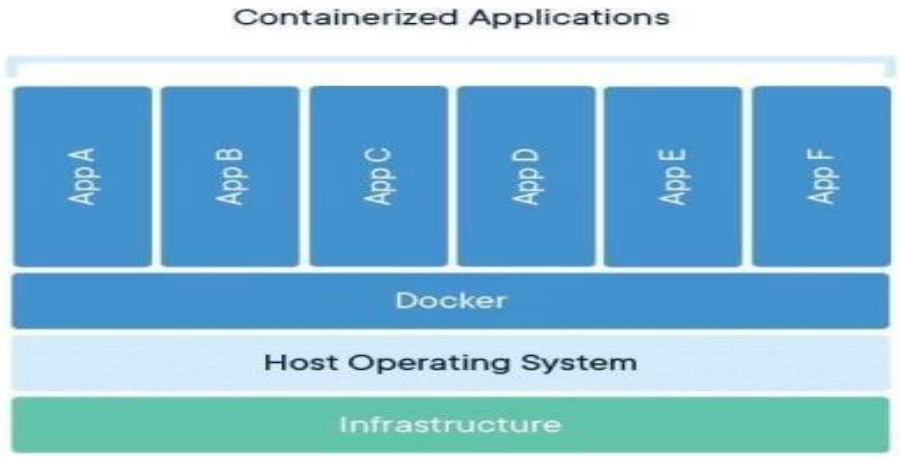



Figure 20: Architecture de Conteneurisation

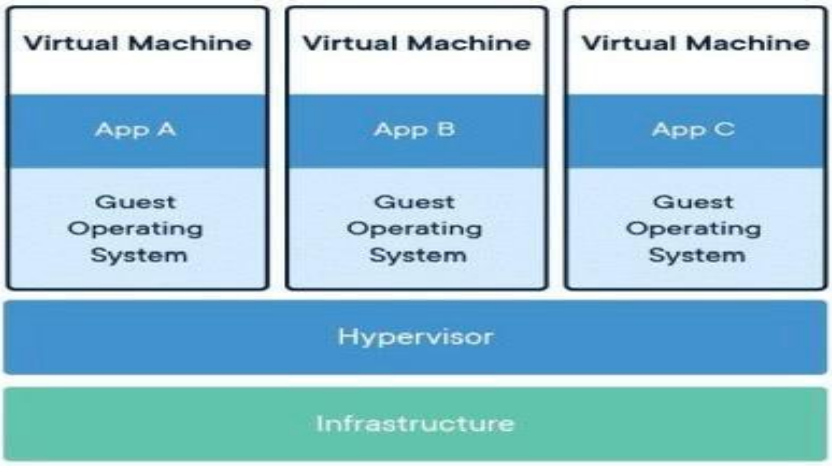



Figure 21: Architecture de Virtualisation

Avant, pour déployer une application, on l'installait directement sur le système d'un serveur (l'hôte), en utilisant les gestionnaires de paquets de l'OS. Le problème, c'est que tout était lié : le code, les configurations, les bibliothèques… Et dès qu'on touchait à quelque chose, ça pouvait casser le reste.

On pouvait aussi créer des images de machines virtuelles pour faciliter les déploiements et les retours arrière, mais les VMs sont lourdes, lentes à lancer,

et pas faciles à déplacer d'un environnement à un autre.

Aujourd'hui, la méthode moderne repose sur les conteneurs, qui utilisent une forme de virtualisation plus légère, directement au niveau du système d'exploitation (et non du matériel comme les VMs).

Chaque conteneur est isolé, avec son propre système de fichiers, ses processus, et ses ressources. Ils sont plus simples à créer que les VMs, plus rapides, et surtout portables : on peut les faire tourner sur n'importe quel OS ou cloud, sans modifier l'application

**À noter :**

Docker ne remplace pas complètement les VMs. En réalité, on combine souvent les deux :

- Les machines virtuelles servent à créer des environnements de base (serveurs virtuels).
- Docker vient ensuite isoler les applications à l'intérieur de ces environnements.

### 3.1.4. Docker networking

Le réseau Docker sert à connecter les conteneurs entre eux, mais aussi à l'extérieur (internet ou autre machine). Grâce à cela, les conteneurs peuvent communiquer entre eux et aussi échanger avec le système hôte (Docker Host).

Docker s'appuie sur un modèle appelé CNM (Container Network Model) pour gérer le réseau. Ce modèle définit les étapes standards à suivre pour configurer un réseau pour les conteneurs, peu importe le pilote réseau utilisé (bridge, host, overlay, etc.).

En résumé, Docker Networking rend possible la communication entre les conteneurs, entre les conteneurs et le système hôte, et même avec l'extérieur, tout en gardant les connexions bien organisées et contrôlées.

**Les différents types de réseau:**

#### 3.1.4.1. Bridge

Le réseau de bridge est un réseau par défaut créé automatiquement

lorsque vous déployez un conteneur. Le réseau de pont utilise un pont logiciel qui permet aux conteneurs connectés au même réseau de pont de communiquer. Réseaux de pont utilisés sur des conteneurs qui s'exécutent sur le même hôte de démon Docker. Le réseau de pont crée un réseau interne privé isolé vers l'hôte afin que les conteneurs de ce réseau puissent communiquer.

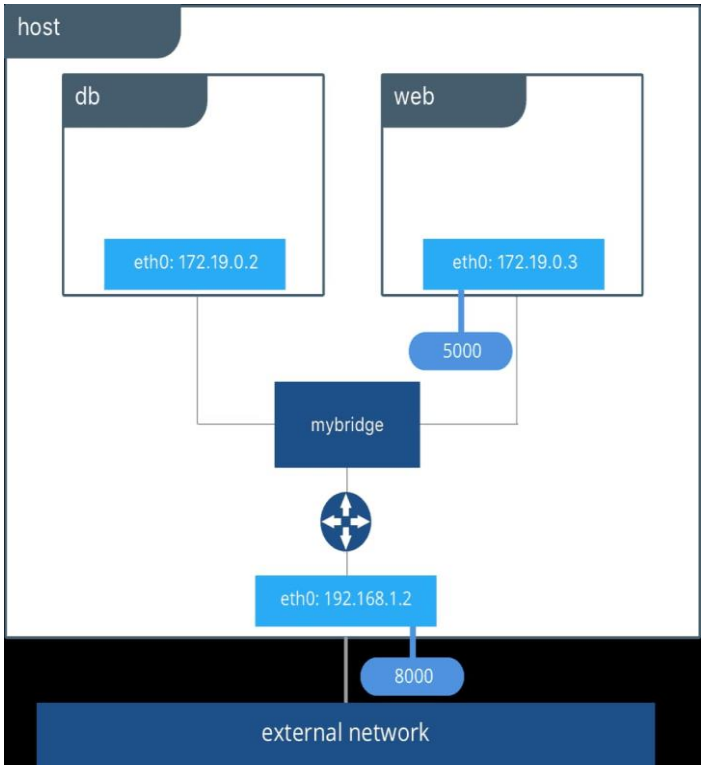



Figure 22: Réseau Bridge

### 3.1.4.2. Host

Cela supprime toute isolation réseau entre l'hôte docker et les conteneurs docker. La mise en réseau en mode hôte peut être utile pour optimiser les performances. Il ne nécessite pas de traduction d'adresses réseau (NAT).

Note:

Le pilote de mise en réseau de l'hôte ne fonctionne que sur les hôtes Linux et n'est pas pris en charge sur Docker Desktop pour Mac, Docker Desktop pour Windows ou Docker EE pour Windows Server.

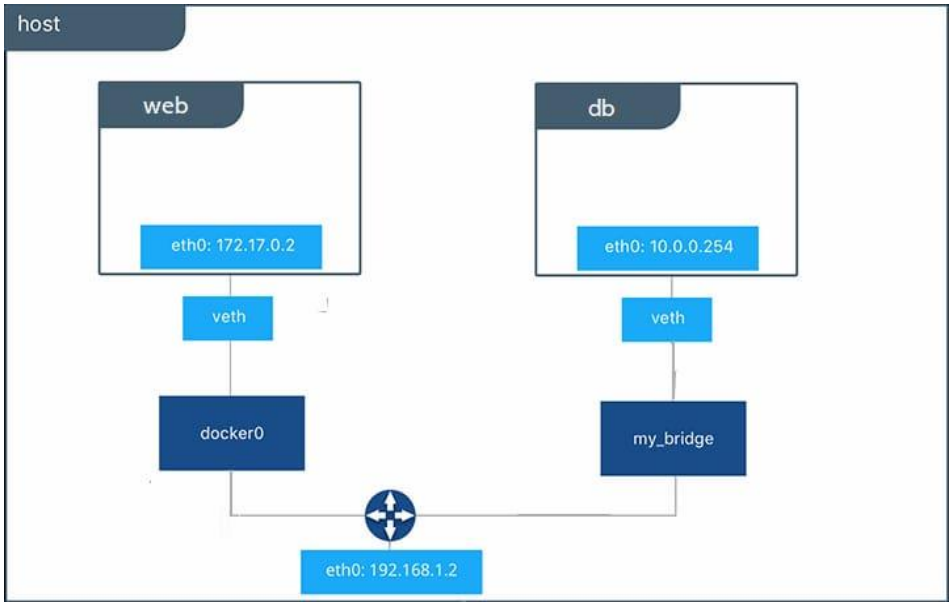



Figure 23 : Reseaux host

### 3.1.4.3. Overlay

Le réseau de superposition est utilisé si le conteneur sur le nœud A veut communiquer avec le nœud B, puis pour établir une communication entre eux, nous utilisons le réseau de superposition. La mise en réseau de superposition utilise VXLAN pour créer un réseau de superposition.

Cela présente l'avantage d'offrir une portabilité maximale sur divers réseaux cloud et sur site. Par défaut, le réseau de superposition est chiffré à l'aide de l'algorithme AES.

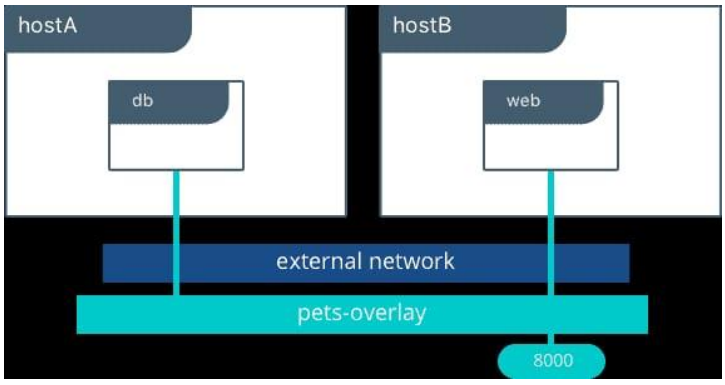



Figure 24: Réseau Overlay

### 3.1.4.4. Null

C'est le type de réseau idéal, si vous souhaitez interdire toute communication interne et externe avec votre conteneur, car votre conteneur sera dépourvu de toute interface réseau.

### 3.1.4.5. Macvlan

Le réseau Macvlan est utilisé pour connecter des applications directement au réseau physique. En utilisant le pilote réseau macvlan pour attribuer une adresse MAC à chaque conteneur, vous pouvez également disposer d'une pile TCP/IP complète. Ensuite, le démon Docker achemine le trafic vers les conteneurs par leurs adresses MAC. Vous pouvez isoler vos réseaux macvlan à l'aide de différentes interfaces réseau physiques. Ceci est utilisé dans les applications héritées qui nécessitent une adresse MAC.

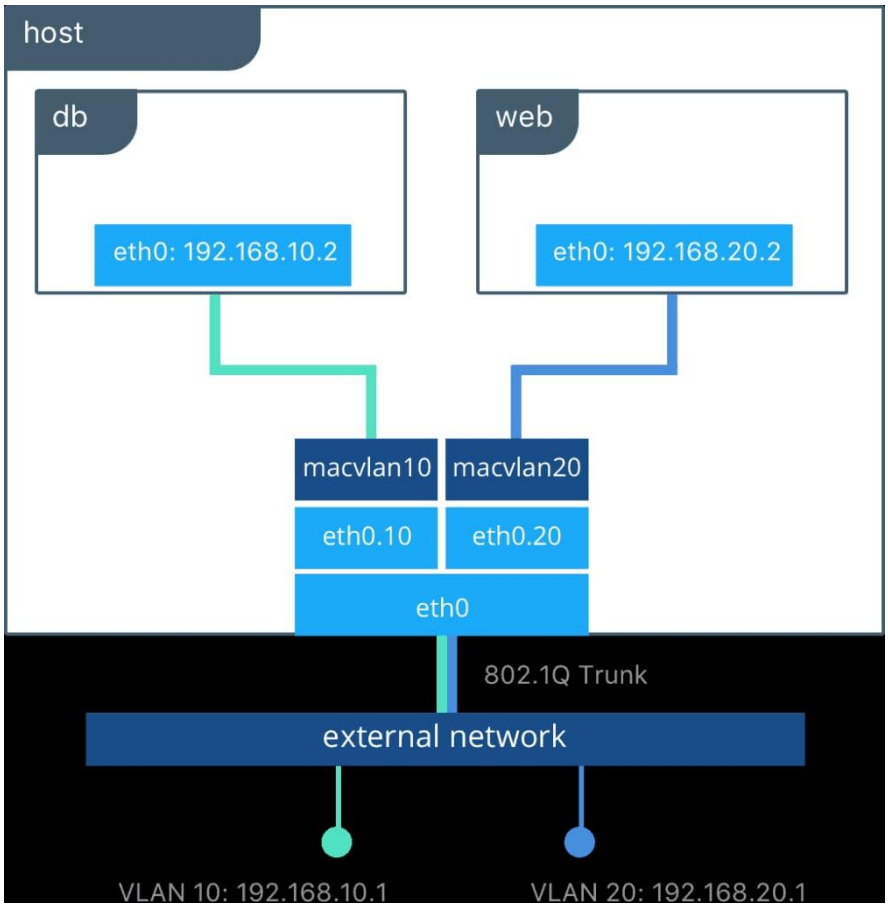



Figure 25: Réseau Macvlan

### 3.1.5. Quelques commandes de base pour Docker

#### 3.1.5.1. Commandes de gestion des conteneurs

- Docker run -it <image>: Exécute un conteneur à partir d'une image et ouvre une session interactive.
- Docker ps: Liste les conteneurs en cours d'exécution.
- Docker ps -a: Liste tous les conteneurs, y compris ceux qui sont arrêtés.
- Docker stop <conteneur>: Arrête un conteneur.
- Docker start <conteneur>: Démarre un conteneur arrêté.
- Docker rm <conteneur>: Supprime un conteneur.

#### 3.1.5.2. Commandes de gestion des images

- Docker images: Liste les images Docker disponibles.
- Docker pull <image>: Télécharge une image Docker depuis un registre.
- Docker build -t <image> : Construit une image Docker à partir d'un Dockerfile dans le répertoire courant.
- Docker rmi <image>: Supprime une image Docker.

#### 3.1.5.3. Commandes de gestion des volumes

- Docker volume create <volume>: Crée un volume Docker.
- Docker volume ls: Liste les volumes Docker disponibles.
- Docker volume rm <volume>: Supprime un volume Docker.

#### 3.1.5.4. Commandes de gestion des réseaux

- Docker network create <réseau>: Crée un réseau Docker.
- Docker network ls: Liste les réseaux Docker disponibles.
- Docker network rm <réseau>: Supprime un réseau Docker.

#### 3.1.5.5. Commandes diverses

- Docker logs <conteneur>: Affiche les journaux d'un conteneur.
- Docker exec -it <conteneur> <commande>: Exécute une commande dans un conteneur en cours d'exécution.
- Docker inspect <conteneur>: Affiche des informations détaillées sur un conteneur.
- Docker inspect <conteneur>: Affiche des informations détaillées sur un conteneur.

#### 3.1.5.6. Voici d'autres commandes Docker

- Docker run -it ubuntu /bin/bash: Exécute un conteneur Ubuntu et ouvre

une session interactive.

- Docker run -d -p 8080:80 nginx: Exécute un conteneur Nginx en mode détaché et mappe le port 8080 de l'hôte au port 80 du conteneur.
- Docker build -t mon-image .: Construit une image Docker à partir d'un Dockerfile dans le répertoire courant et la nomme "mon-image".

### 3.1.6. Conclusion

Dans cette partie, nous avons étudié Docker, la solution de conteneurisation la plus largement adoptée dans l'industrie. Docker permet d'encapsuler une application et toutes ses dépendances dans un environnement isolé, portable et léger, facilitant ainsi le développement, le test et le déploiement d'applications sur différentes infrastructures.

Nous avons présenté ses principales fonctionnalités, notamment la gestion des images, des conteneurs, des volumes et des réseaux. L'utilisation de Docker s'inscrit pleinement dans les pratiques DevOps modernes, en accélérant les cycles de développement tout en assurant la cohérence entre les environnements.

L'étude de ses alternatives (Podman, CRI-O, Containerd, LXD) a également permis de mieux comprendre les cas d'usage spécifiques et les choix techniques possibles en fonction des contraintes de sécurité, de performance ou d'intégration avec des orchestrateurs comme Kubernetes.

Docker représente ainsi une base incontournable pour quiconque souhaite construire des systèmes modulaires, évolutifs et facilement déployables, que ce soit en local, en cloud ou dans des environnements hybrides.

## 3.2. Kubernetes

### 3.2.1. Introduction

Kubernetes (ou K8s) est une plateforme open source qui sert à gérer automatiquement les conteneurs d'applications. Elle permet de déployer, surveiller, mettre à jour et faire tourner des applications conteneurisées sur un ensemble de machines (appelé cluster).

Créé par Google, Kubernetes aide les développeurs à emballer leurs

applications avec tout ce qu'il faut (dépendances, configurations...) et à les déployer rapidement, peu importe que ce soit sur du cloud privé, public ou hybride.

Côté réseau, Kubernetes permet à tous les composants et conteneurs de communiquer entre eux facilement grâce à une architecture réseau à plat. Cela évite d'avoir à faire des réglages compliqués de ports. En gros, c'est une solution pratique pour faire tourner des systèmes distribués sur plusieurs machines, sans prise de tête avec le réseau.

### 3.2.2. Définition de Kubernetes

Kubernetes (souvent abrégé K8s) est une plateforme open-source développée par Google pour automatiser le déploiement, la mise à l'échelle et la gestion des applications en conteneurs (comme ceux de Docker). Il orchestre des groupes de conteneurs à travers un cluster de machines.

### 3.2.3. Avantages de Kubernetes

- **Orchestration automatisée des conteneurs**

Kubernetes gère le cycle de vie des conteneurs (déploiement, redémarrage, remplacement, etc.) sans intervention manuelle, garantissant un fonctionnement continu et fluide des applications.

- **Haute disponibilité**

En cas de panne d'un conteneur ou d'un nœud, Kubernetes détecte automatiquement l'incident et redéploie les services affectés pour maintenir la disponibilité de l'application.

- **Mise à l'échelle simplifiée**

Le cluster peut être mis à l'échelle manuellement ou automatiquement en fonction de la charge, permettant une adaptation dynamique aux besoins de performance.

- **Réseau interne sécurisé**

Kubernetes établit un réseau virtuel isolé entre les conteneurs, avec la possibilité de contrôler finement les communications internes via des politiques réseaux.

- **Portabilité et flexibilité**

Kubernetes fonctionne de manière cohérente sur des infrastructures locales, cloud (AWS, Azure, GCP, etc.), ou hybrides, garantissant une portabilité maximale des applications conteneurisées.

### 3.2.4. Architecture de Kubernetes

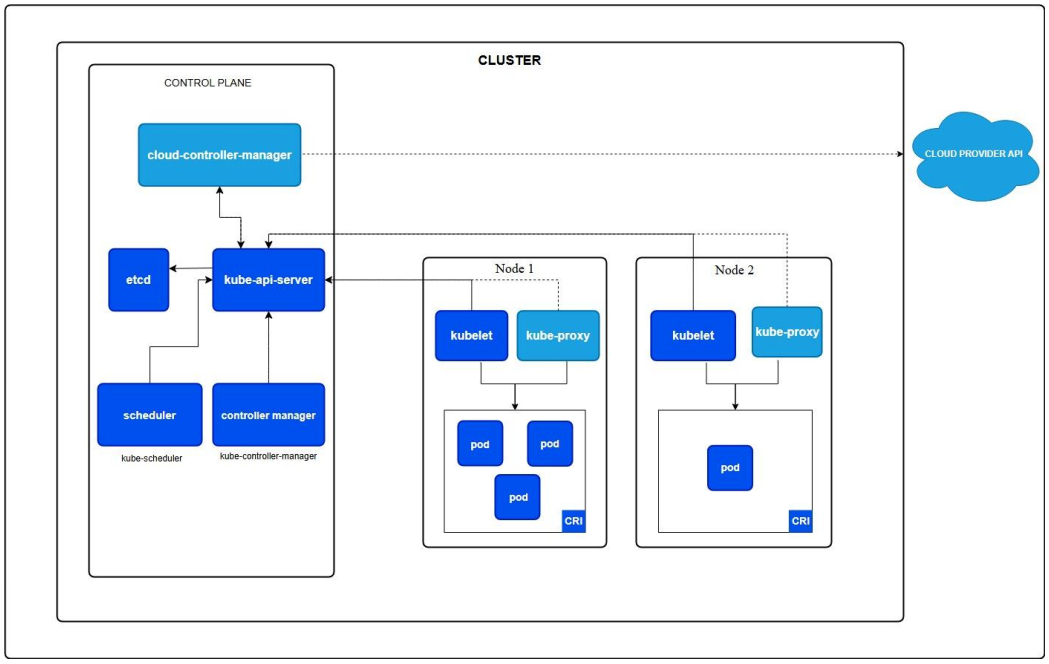



Figure 26: Architecture Kubernetes

#### 3.2.4.1. Les Pods

Un pod est l'unité de base dans Kubernetes. Il peut contenir un ou plusieurs conteneurs qui partagent les mêmes ressources (comme le processeur, la mémoire, le réseau...).

Tous les conteneurs d'un pod sont toujours lancés ensemble sur le même nœud, ce qui leur permet de communiquer facilement entre eux.

#### 3.2.4.2. Le Cluster Kubernetes et les Nœuds

Un cluster Kubernetes est composé de plusieurs machines appelées nœuds. Ces nœuds sont divisés en deux types :

- Les nœuds master (ou plan de contrôle) : ils gèrent l'état global du cluster et envoient les instructions.
- Les nœuds workers : ce sont eux qui exécutent les applications dans les

pods, selon ce que leur dit le master.

#### 3.2.4.3. Le Data Plane

Le data plane correspond à tous les nœuds workers du cluster. Il est responsable de faire tourner les conteneurs et d'exécuter les instructions reçues du master.Il se compose de:

- API-server: Le serveur API est le point d'entrée principal du cluster Kubernetes. C'est lui qui reçoit toutes les requêtes, que ce soit des utilisateurs, d'autres services internes (comme le planificateur), ou des outils comme kubectl. Il sert d'intermédiaire pour toute la communication dans le cluster. Par exemple, si le planificateur veut envoyer un pod sur un nœud, il passe par le serveur API, et non directement.
- scheduler: Le planificateur décide où exécuter les pods dans le cluster. Il regarde les ressources disponibles (CPU, RAM…) sur chaque nœud, et place les pods là où ça convient le mieux, en respectant les règles définies par l'utilisateur. Par exemple, si on déploie une nouvelle application, le planificateur choisit automatiquement le meilleur nœud pour l'exécuter.
- etcd: etcd est une base de données clé-valeur utilisée par Kubernetes pour stocker toutes les données importantes du cluster : les configurations, l'état actuel des nœuds, les informations réseau, etc. Il garantit que toutes ces données soient persistantes (elles ne disparaissent pas après un redémarrage) et synchronisées entre tous les composants.
- controller manager: Le controller manager surveille en permanence l'état du cluster pour que tout fonctionne comme prévu. Il compare l'état souhaité (défini dans les fichiers de configuration) à l'état réel. Par exemple, si on demande 2 copies d'un pod et qu'une seule tourne, le contrôleur en lance une deuxième. S'il y en a 3, il en supprime une. Il s'assure toujours que tout reste conforme.

#### 3.2.4.4. Le Control Plane

Le control plane, lui, regroupe les nœuds master. C'est lui qui prend les décisions : il planifie les pods, surveille l'état du cluster et applique les règles de

configuration. Il se compose de:

- Kubelet : Le Kubelet est un petit programme installé sur chaque nœud worker. Il est responsable de faire tourner les pods selon les instructions envoyées par le control plane. En plus de ça, Kubelet surveille l'état des pods et remonte des informations (metrics) vers le master. Cela permet au scheduler de prendre des décisions basées sur l'état réel du nœud (CPU dispo, mémoire, etc.).
- Kube-proxy : Le Kube-proxy s'occupe du routage réseau dans le cluster. Il redirige les requêtes vers les bons pods et permet d'exposer les services à l'extérieur du cluster. On peut dire qu'il agit un peu comme un équilibreur de charge (load balancer).
- Interface réseau pour conteneurs : Le CNI (Container Network Interface) permet de créer des réseaux virtuels dans le cluster. Grâce à lui, les pods peuvent communiquer entre eux, même s'ils sont sur des nœuds différents. Chaque pod reçoit une IP locale, et une interface réseau virtuelle

### 3.2.5. Commandes Kubernetes de Base

#### 3.2.5.1. Gestion des conteneurs (pods)

- **kubectl get pods :** Cette commande permet d'afficher la liste des pods actuellement actifs dans le cluster Kubernetes. Elle donne une vue d'ensemble de l'état des applications déployées.
- **kubectl describe pod <nom> :** Elle fournit des informations détaillées sur un pod spécifique, telles que ses événements récents, sa configuration, son statut ou encore les erreurs éventuelles.
- **kubectl delete pod <nom> :** Utilisée pour supprimer un pod donné. Si le pod appartient à un déploiement (Deployment), Kubernetes le recréera automatiquement pour garantir la disponibilité.
- **kubectl apply -f fichier.yaml :** Cette commande permet de créer ou de mettre à jour des ressources Kubernetes en appliquant un fichier de configuration au format YAML. Elle est idéale pour gérer l'infrastructure de manière déclarative.

#### 3.2.5.2. Gestion des déploiements

- **kubectl create deployment <nom> --image=<image> : Cette** commande permet de créer un déploiement Kubernetes à partir d'une image Docker. Elle initialise automatiquement un pod géré par un contrôleur de type Deployment.
- **kubectl get deployments :** Elle permet d'afficher la liste des déploiements existants dans le cluster, ainsi que leur état (nombre de pods disponibles, en cours de création ou d'arrêt, etc.).
- **kubectl scale deployment <nom> --replicas=N :** Utilisée pour modifier dynamiquement le nombre de pods associés à un déploiement. Cela permet de mettre à l'échelle une application en fonction des besoins.
- **kubectl rollout restart deployment <nom> :** Cette commande redémarre proprement un déploiement en redéployant tous les pods qu'il gère. Elle est utile lors d'une mise à jour de configuration ou pour appliquer une nouvelle version de l'image.

#### 3.2.5.3. Gestion des volumes (stockage)

Kubernetes permet de déclarer des volumes dans un pod à l'aide de fichiers YAML. Voici un exemple simple d'un volume de type hostPath, qui monte un dossier du système hôte dans le conteneur.

**Exemple de déclaration dans un pod :**

yaml
Copier
Modifier

```
apiVersion: v1
kind: Pod
metadata:
  name: mon-pod
spec:
  containers:
  - name: mon-container
    image: nginx
    volumeMounts:
    - name: mon-volume
      mountPath: /app/data
  volumes:
  - name: mon-volume
```

```
  hostPath:
    path: /data
    type: DirectoryOrCreate
```

**Explication :**

**volumeMounts:** indique où le volume sera monté dans le conteneur (/app/data dans cet exemple).

**hostPath:** monte un dossier du système de fichiers de l'hôte (ici /data) dans le pod.

**type: DirectoryOrCreate :** crée le dossier /data sur l'hôte s'il n'existe pas déjà.

#### 3.2.5.4. Gestion des services (réseau)

- **kubectl expose deployment <nom> --type=NodePort --port=80**

Cette commande permet de créer un service à partir d'un déploiement existant. En choisissant le type NodePort, le service expose le port 80 du conteneur sur un port aléatoire du nœud, rendant l'application accessible depuis l'extérieur du cluster.

- **kubectl get svc**

Elle affiche la liste de tous les services présents dans le cluster, avec leurs noms, types, adresses IP internes et ports d'accès. C'est une commande essentielle pour vérifier que les services sont bien créés et accessibles.

### 3.2.6. Conclusion

À travers cette partie, nous avons exploré les fondements de Kubernetes, son architecture, ses commandes essentielles ainsi que ses mécanismes de déploiement, de mise à l'échelle et de gestion des ressources. Kubernetes s'impose aujourd'hui comme la solution de référence pour l'orchestration de conteneurs, grâce à sa robustesse, sa flexibilité, et sa capacité à automatiser l'ensemble du cycle de vie des applications.

Sa compatibilité avec les environnements multi-cloud, son système de services résilient, ainsi que son modèle déclaratif via les fichiers YAML en font une plateforme idéale pour répondre aux exigences modernes des architectures distribuées et DevOps.

Cette étude nous a permis de mieux comprendre l'intérêt d'un orchestrateur tel que Kubernetes dans la gestion de systèmes conteneurisés à grande échelle, tout en mettant en pratique les notions clés au travers de la création et de la configuration d'un cluster fonctionnel.

## 3.3. Conclusion

Cette première partie nous a permis de poser les bases techniques nécessaires à la mise en place d'un cluster Kubernetes en explorant les technologies fondamentales de la conteneurisation et de la virtualisation. À travers l'étude détaillée de Docker, nous avons compris comment les conteneurs facilitent le déploiement d'applications de manière légère, portable et cohérente. Nous avons également analysé ses alternatives afin d'élargir notre compréhension des solutions existantes et de mieux orienter nos choix techniques.

L'introduction à Kubernetes nous a ensuite permis de saisir son rôle central dans l'orchestration des conteneurs, en automatisant la gestion, le redémarrage, la montée en charge et la résilience des applications dans un environnement distribué. Sa richesse fonctionnelle en fait une solution incontournable pour la gestion d'infrastructures modernes.

Enfin, nous avons abordé les technologies de virtualisation qui restent complémentaires à la conteneurisation, notamment dans les environnements de développement, de test ou de simulation.

Cette partie constitue ainsi un socle solide sur lequel repose la conception et la réalisation du cluster Kubernetes présentées dans la deuxième partie du projet. Elle nous a permis d'acquérir une vision globale, structurée et comparative des outils à mobiliser pour construire une infrastructure performante, évolutive et adaptée aux standards actuels de l'ingénierie logicielle.

# Partie II: Conception et Réalisation du Cluster Kubernetes

Cette partie est consacrée à la conception et modélisation du projet il va détailler les différentes étapes de conception et de réalisation du projet.

# *1. Chapitre 1 : Conception de l'architecture du Cluster Kubernetes*

## 1.2. Objectif du cluster

L'objectif principal de notre cluster Kubernetes est de mettre en place un **environnement de test local** permettant de simuler le déploiement et la gestion d'applications conteneurisées. À travers cette infrastructure distribuée, composée de plusieurs machines virtuelles interconnectées, nous cherchons à **expérimenter le fonctionnement concret de Kubernetes** dans un cadre proche des environnements de production. Ce cluster nous permettra d'approfondir des aspects essentiels tels que le **déploiement de pods**, la **gestion du réseau**, **l'équilibrage de charge**, ainsi que l'**orchestration des services**, en vue de mieux maîtriser les mécanismes sous-jacents à cette plateforme.

## 1.3. Choix du type de cluster

Pour ce projet, nous avons opté pour la création d'un cluster Kubernetes local, déployé sur VirtualBox à l'aide de machines virtuelles Ubuntu Server 22.04.

Ce choix s'appuie sur plusieurs considérations :

- L'absence de nécessité d'un environnement cloud pour nos tests et expérimentations.
- La facilité offerte par VirtualBox pour créer, configurer et interconnecter plusieurs machines virtuelles de manière simple et flexible.
- La stabilité et la compatibilité d'Ubuntu Server 22.04 avec les outils essentiels tels que kubeadm, Docker et le réseau Flannel, garantissant ainsi une infrastructure fiable et cohérente pour le cluster.Pour ce projet, nous avons choisi de créer un cluster Kubernetes en local sur VirtualBox, avec des machines Ubuntu Server 22.04.

Ce choix a été fait pour plusieurs raisons :

- Nous n'avions pas besoin d'un cluster dans le cloud.
- VirtualBox permet de facilement créer, configurer et relier plusieurs

machines entre elles.

- Ubuntu est un système stable, compatible avec kubeadm, Docker et Flannel.

## 1.4. Nombre de nœuds et leur rôle

Nous avons configuré un total de quatre machines virtuelles pour constituer notre cluster Kubernetes :

- **Un nœud Master :** machine principale chargée de gérer le cluster, incluant le plan de contrôle, la planification des tâches et la supervision des ressources.
- **Trois nœuds Workers :** machines dédiées à l'exécution des pods, c'est-à-dire des applications et services déployés sur le cluster.

Chaque machine est configurée avec une adresse IP privée statique, assurant ainsi une communication stable et fiable entre les différents nœuds du cluster.

## 1.5. Architecture réseau

Pour la configuration réseau, nous avons opté pour un réseau privé de type "Host-Only" sous VirtualBox. Ce type de réseau permet d'isoler les machines virtuelles du réseau externe tout en leur offrant la possibilité de communiquer librement entre elles.

Afin de gérer la communication entre les pods à travers les différents nœuds du cluster, nous avons installé Flannel en tant que plugin CNI (Container Network Interface). Flannel facilite la création d'un réseau virtuel qui attribue des adresses IP aux pods, assurant ainsi leur connectivité inter-nœuds. Ce choix s'est avéré adapté à notre environnement local grâce à sa simplicité de configuration et sa fiabilité.

Les adresses IP des machines virtuelles ont été attribuées de manière statique pour garantir une connexion stable, par exemple :

**Master :** 192.168.60.12
**Worker1** : 192.168.60.21
**Worker2 :** 192.168.60.22
**Worker3 :** 192.168.60.23

## 1.6. Schema d'architecture

Le schéma ci-dessous illustre l'architecture globale de notre cluster Kubernetes local déployé sur VirtualBox. Il met en évidence la répartition des rôles entre le nœud Master, responsable du plan de contrôle, et les trois nœuds Workers, qui hébergent les pods applicatifs. On y distingue également la configuration réseau en mode **Host-Only**, garantissant la communication interne sécurisée entre les machines virtuelles.

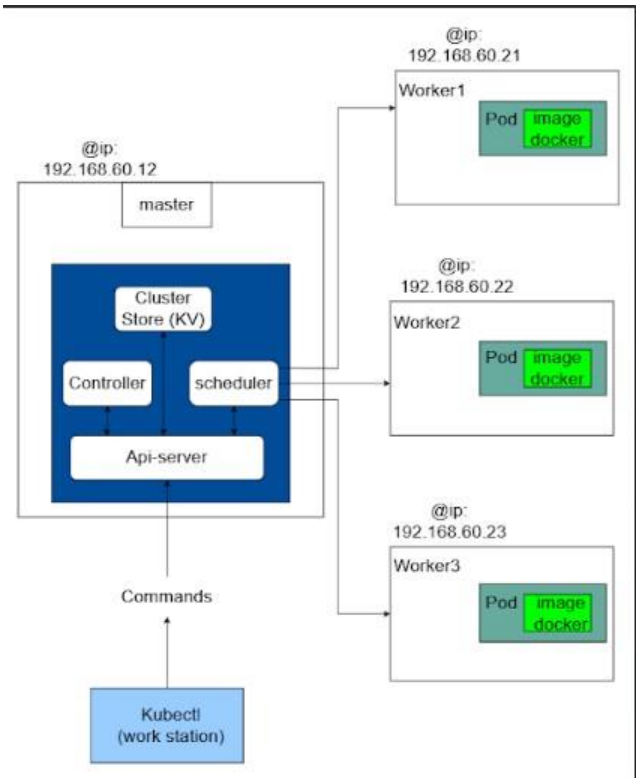



Figure 27 : Architecture du cluster

# 2. Chapitre 2 : Réalisation du cluster kubernetes

## Les étapes de la mise en place d'un cluster kubernetes

### 2.1. Étape 1 : Installation de VirtualBox et Configuration des Machines Virtuelle

Dans un premier temps, nous avons installé VirtualBox, un logiciel de virtualisation open source, afin de créer notre cluster Kubernetes. VirtualBox nous offre la possibilité de simuler plusieurs machines virtuelles sur une seule machine physique, ce qui constitue une solution économique et pratique pour tester et configurer un environnement Kubernetes sans recourir à une infrastructure physique coûteuse.

Une fois VirtualBox installé, nous avons procédé à la création des machines virtuelles indispensables au fonctionnement de notre cluster

#### 2.1.1. Création des Machines Virtuelles

Nous avons configuré un total de huit machines virtuelles dans VirtualBox afin de constituer notre cluster Kubernetes :

- **Trois nœuds Master :** Ces nœuds centraux sont chargés de l'orchestration du cluster, incluant la gestion du plan de contrôle, la planification et la supervision des ressources.
- **Quatre nœuds Worker :** Ces nœuds exécutent les applications et services déployés sous forme de pods.
- **Un Load Balancer** : Cette machine assure la répartition du trafic réseau entre les différents nœuds Masters, garantissant ainsi la haute disponibilité et la résilience du cluster.

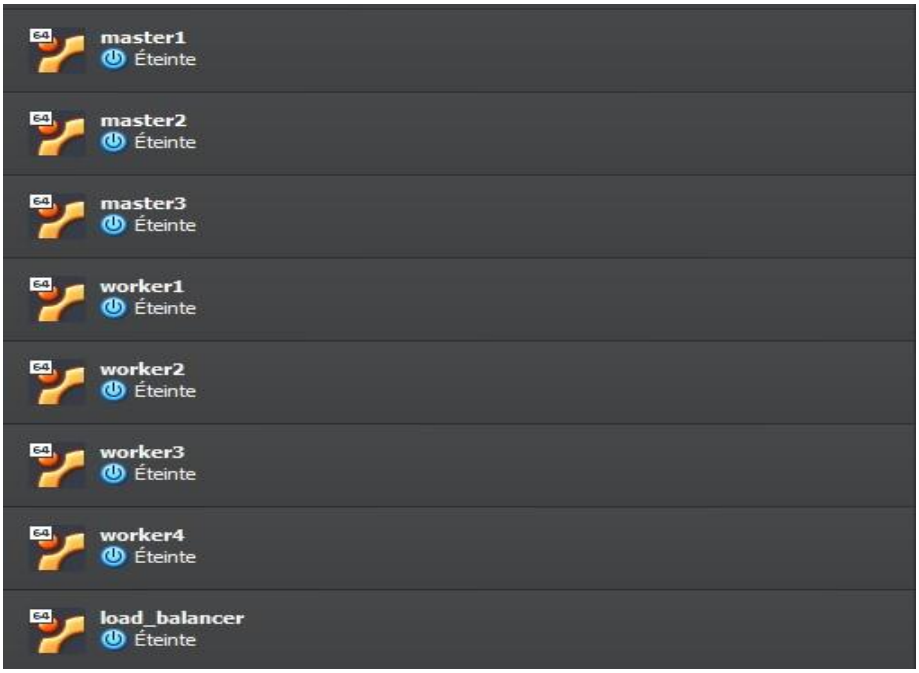



Figure 28 : Les machines virtuelles

### 2.1.2. Installation du Système d'Exploitation

Sur chaque machine virtuelle, nous avons installé Ubuntu Server 22.04 LTS, une version stable, fiable et largement adoptée dans les environnements cloud et conteneurisés. Cette distribution a été choisie pour sa compatibilité avec les outils nécessaires à la mise en place d'un cluster Kubernetes, tels que kubeadm, Docker et les plugins CNI.

L'installation a été effectuée à partir d'une image ISO officielle, téléchargée directement depuis le site d'Ubuntu, garantissant ainsi l'authenticité et la sécurité du système.

### 2.1.3. Configuration Réseau dans VirtualBox

Afin de permettre la communication entre les différentes machines virtuelles, nous avons configuré un réseau de type **Host-Only** dans VirtualBox. Ce mode de réseau offre une connexion privée entre les machines virtuelles, leur permettant d'échanger des données entre elles tout en restant isolées du réseau externe, ce qui renforce la sécurité et le contrôle de l'environnement de test.

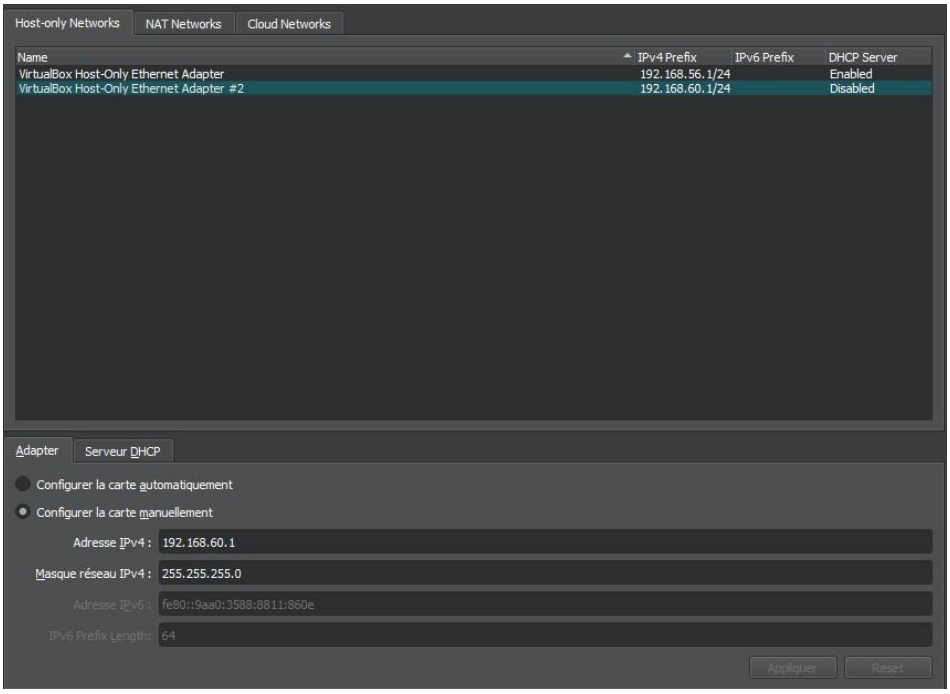



Figure 29: Configuration du Host-Only Networks

Nous avons également attribué des adresses IP statiques à chaque machine virtuelle afin de simplifier leur identification et leur gestion au sein du cluster. Cette configuration garantit une communication stable et prévisible entre les nœuds.

Les adresses IP ont été définies comme suit :

- **Load Balancer :** 192.168.60.10
- **Nœuds Master :** de 192.168.60.11 à 192.168.60.13
- **Nœuds Worker :** de 192.168.60.21 à 192.168.60.24

Cette organisation logique permet de distinguer clairement les rôles des machines et facilite les opérations d'administration et de surveillance du cluster.

La commande qui attribue les adresses IP est :

```
sudo nmcli connection add type ethernet ifname enp0s8  con-name enp0s8
ip4 192.168.60.11/24 gw4 '' +ipv4.dns    "8.8.8.8 8.8.4.4"
```

```
master1@master1-VirtualBox:~$ sudo nmcli connection add type ethernet ifname enp
0s8 ip4 192.168.60.11/24 gw4 '' +ipv4.dns "8.8.8.8 8.8.4.4"
```

Figure 30: Commande d'attribution des adresses IP

### 2.1.4. Vérification de la Connectivité

Après avoir configuré le réseau, nous avons procédé à des tests de connectivité entre les différentes machines virtuelles en utilisant la commande ping. Cette étape nous a permis de confirmer que toutes les VMs pouvaient communiquer correctement entre elles, assurant ainsi un fonctionnement cohérent du cluster.

La commande utilisée pour vérifier la connexion est la suivante :

| **Ping [adresse IP de destination]** |
|---|

```
master1@master1-VirtualBox:~$ ping -c 5 192.168.60.12
PING 192.168.60.12 (192.168.60.12) 56(84) bytes of data.
64 bytes from 192.168.60.12: icmp_seq=1 ttl=64 time=3.99 ms
64 bytes from 192.168.60.12: icmp_seq=2 ttl=64 time=2.65 ms
64 bytes from 192.168.60.12: icmp_seq=3 ttl=64 time=0.968 ms
64 bytes from 192.168.60.12: icmp_seq=4 ttl=64 time=2.94 ms
64 bytes from 192.168.60.12: icmp_seq=5 ttl=64 time=3.08 ms

--- 192.168.60.12 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4214ms
rtt min/avg/max/mdev = 0.968/2.723/3.992/0.986 ms
```

Figure 31 : Commande de vérification de la connectivité

## 2.2. Étape 2 : Installation et configuration des composants Kubernetes

Après avoir configuré nos machines virtuelles et le réseau, nous avons procédé à l'installation des outils nécessaires au déploiement et à la gestion de notre cluster Kubernetes. Voici les différentes étapes suivies :

### 2.2.1. Mise à jour de la liste des paquets

Pour garantir l'accès aux versions les plus récentes des paquets logiciels, la première étape a consisté à actualiser la base de données des paquets avec la commande suivante :

```
sudo apt-get update
```

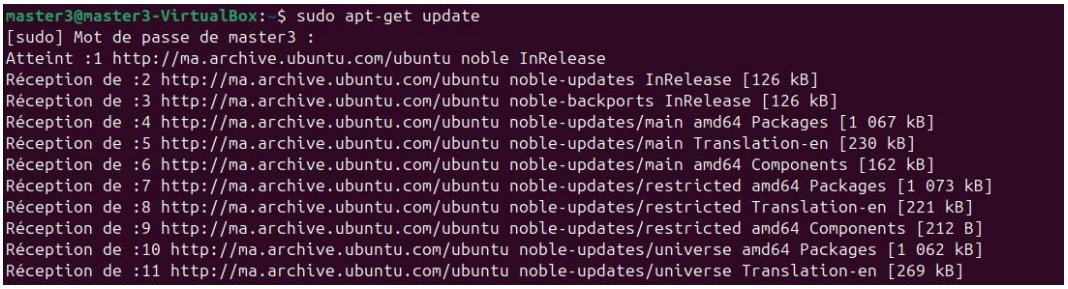



Figure 32 : Commande de Mise à jour de la liste des paquets

Cette opération interroge les dépôts configurés sur le système pour récupérer les dernières informations sur les logiciels disponibles, assurant ainsi que toute installation ultérieure se fera avec des versions à jour.

### 2.2.2. Installation des paquets essentiels

Avant d'ajouter le dépôt officiel Kubernetes, il est nécessaire d'installer plusieurs outils permettant de gérer les connexions sécurisées, les certificats et les téléchargements :

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
```

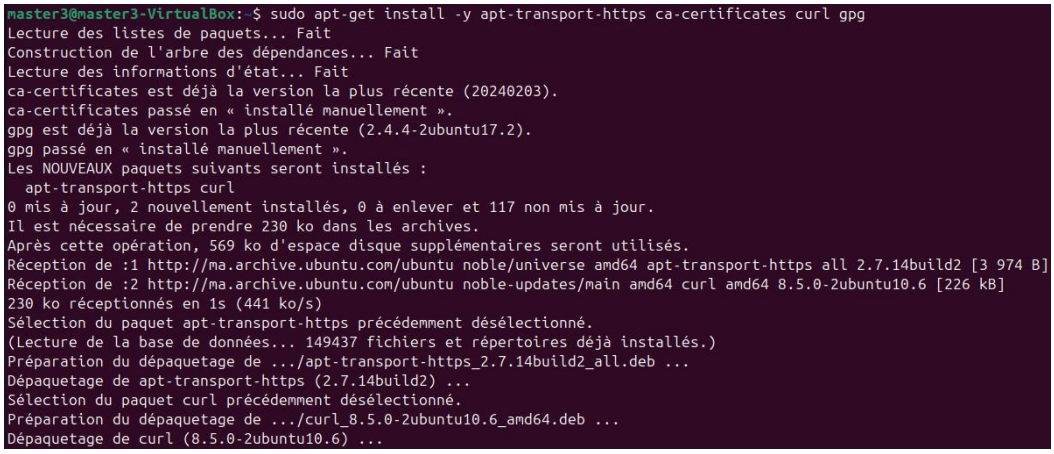



Figure 33 : Installation des paquets essentiels

- **apt-transport-https :** Permet au gestionnaire de paquets APT de télécharger les paquets via HTTPS, garantissant la sécurité des transferts.
- **ca-certificates :** Assure la validation des certificats SSL.
- **curl :** Utilisé pour récupérer des fichiers à partir d'Internet.
- **gpg :** Utilisé pour vérifier la signature des paquets et garantir leur authenticité. Ces outils nous permettent de gérer les connexions sécurisées, de télécharger des fichiers, et de vérifier l'authenticité des paquets."

### 2.2.3. Importation de la Clé GPG du Dépôt Kubernetes

Pour éviter toute altération malveillante des paquets téléchargés, Kubernetes signe ses dépôts avec une clé GPG. Nous avons importé cette clé de manière sécurisée:

**curl -fsSL https://pkgs.k8s.io/core :/stable:/v1.33/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg**




Figure 34 : Commande de mise à jour de la liste

Cette commande fait deux choses :

- Télécharge la clé GPG officielle du dépôt Kubernetes.

- Convertit cette clé en format binaire (plus sécurisé et performant) et la stocke dans un emplacement dédié (/etc/apt/keyrings/) pour que le gestionnaire de paquets puisse s'en servir lors de la validation des paquets.

### 2.2.4. Ajout du Dépôt Officiel de Kubernetes

Nous avons ensuite ajouté la source officielle des paquets Kubernetes à notre gestionnaire de paquets, en spécifiant que seuls les paquets signés par la clé précédemment importée seront acceptés :

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core :/stable:/v1.33/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

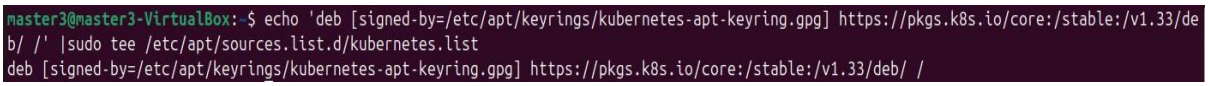



Figure 35 : Commande d'ajout la source officielle des paquets Kubernetes

Cette commande effectue deux actions principales :

- Ajoute le dépôt officiel de Kubernetes à la liste des sources APT.
- Assure que les paquets provenant de ce dépôt sont authentifiés grâce à la clé GPG spécifiée (signed-by).

Une fois cette étape terminée, le système peut rechercher et installer les outils Kubernetes depuis ce dépôt.

### 2.2.5. Mise à jour des paquets après ajout du dépôt

Afin de prendre en compte le nouveau dépôt, nous avons relancé une mise à jour des informations des paquets disponibles :

```
sudo apt-get update
```

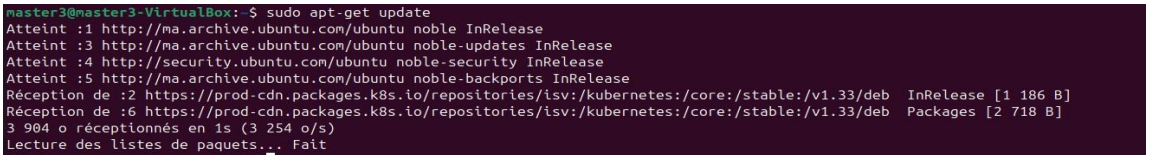


Figure 36 : Mise à jour des paquets

Cette opération est indispensable avant toute installation à partir de ce nouveau dépôt.

### 2.2.6. Configuration des Composants Principaux de Kubernetes

Les composants essentiels pour faire fonctionner un cluster Kubernetes sont kubeadm, kubelet et kubectl. Leur installation diffère légèrement entre les nœuds Master et Worker :

- Sur les nœuds Master, tous les composants sont installés :

**sudo apt-get install -y kubeadm kubelet kubectl**

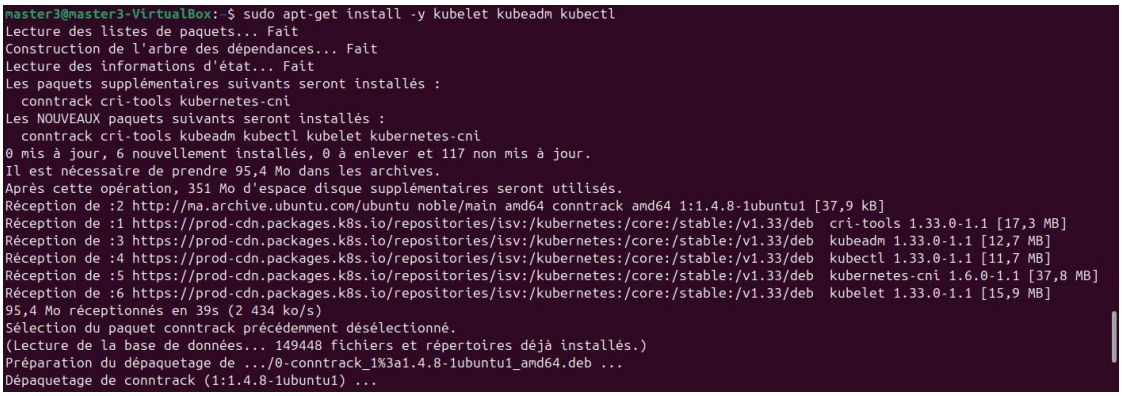



Figure 37 : les nœuds Master

- Sur les nœuds Worker, seuls kubeadm et kubelet sont nécessaires :

**sudo apt-get install -y kubeadm kubelet**




Figure 38 : les nœuds worker

Pour confirmer que l'installation a été réalisée correctement, nous avons vérifié les versions de chaque composant avec :

- kubeadm : Initialise le cluster et ajoute des nœuds.
- kubelet : S'exécute sur chaque nœud et gère les pods et les conteneurs.
- kubectl : Permet d'interagir avec le cluster pour déployer des applications et inspecter les ressources.

**kubeadm version**

**kubectl version --client**

**kubelet –version**

Cette étape permet de s'assurer que les versions installées sont cohérentes et compatibles entre elles.




Figure 39 : Vérification des versions des composants

### 2.2.7. Blocage des Mises à Jour de kubelet, kubeadm et kubectl

Pour garantir la stabilité de notre cluster Kubernetes, nous avons bloqué les mises à jour automatiques des outils kubelet, kubeadm, et kubectl avec la commande suivante :

**Pour les master noeuds**

**sudo apt-mark hold kubelet kubeadm kubectl**

```
master3@master3-VirtualBox:~$ sudo apt-mark hold kubelet kubeadm kubectl
kubelet passé en figé (« hold »).
kubeadm passé en figé (« hold »).
kubectl passé en figé (« hold »).
```

**Pour les workers noeuds :**

**sudo apt-mark hold kubelet kubeadm**

```
worker3@worker3-VirtualBox:~$ sudo apt-mark hold kubeadm kubelet
kubeadm passé en figé (« hold »).
kubelet passé en figé (« hold »).
```

Cette commande empêche les mises à jour non contrôlées qui pourraient entraîner des incompatibilités entre les versions des composants. Cela nous permet de maintenir un environnement stable et de planifier manuellement les mises à jour lorsque nécessaire.

Si on veut annuler  le blocage on peut on peut exécuter cette commande:

**sudo apt-mark unhold kubelet kubeadm kubectl**

## 8. Configuration de kubelet pour un Démarrage Automatique

Pour garantir que le composant kubelet soit opérationnel sur tous les nœuds, nous avons activé et démarré immédiatement son service avec la commande suivante :

**sudo systemctl enable --now kubelet**

```
master3@master3-VirtualBox:~$ sudo systemctl enable --now kubelet
master3@master3-VirtualBox:~$ sudo systemctl is-enabled kubelet
enabled
```

Cette commande effectue deux actions principales :

- Active le service pour qu'il démarre automatiquement au prochain redémarrage du système.

- Démarre immédiatement le service, permettant à kubelet de commencer à gérer les pods et les conteneurs sur le nœud.

Nous avons ensuite vérifié  si le service kubelet est activé pour démarrer automatiquement avec cette commande:

```
sudo systemctl is-enabled kubelet
```

Cette étape est importante, car kubelet est responsable de la communication entre les nœuds et le Master Node, ainsi que de l'exécution des charges de travail.

## 2.3. Étape 3 : Installation de Docker

### 2.3.1. Installation

Docker est l'un des moteurs de conteneurs les plus utilisés, essentiel pour faire tourner les applications conteneurisées dans Kubernetes.

Pour installer Docker sur chaque machine virtuelle, nous avons exécuté la commande suivante :

```
sudo apt install -y docker.io
```

```
master3@master3-VirtualBox:~$ sudo  apt install -y docker.io
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  bridge-utils containerd git git-man liberror-perl pigz runc ubuntu-fan
Paquets suggérés :
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse
  | zfsutils git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
Les NOUVEAUX paquets suivants seront installés :
  bridge-utils containerd docker.io git git-man liberror-perl pigz runc ubuntu-fan
0 mis à jour, 9 nouvellement installés, 0 à enlever et 117 non mis à jour.
Il est nécessaire de prendre 83,0 Mo dans les archives.
Après cette opération, 325 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 http://ma.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65,6 kB]
Réception de :2 http://ma.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33,9 kB]
Réception de :3 http://ma.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.1.12-0ubuntu3.1 [8 599 kB]
Réception de :4 http://ma.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 1.7.24-0ubuntu1~24.04.2 [37,0 MB]
Réception de :5 http://ma.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 26.1.3-0ubuntu1~24.04.1 [32,4 MB]
Réception de :6 http://ma.archive.ubuntu.com/ubuntu noble/main amd64 liberror-perl all 0.17029-2 [25,6 kB]
Réception de :7 http://ma.archive.ubuntu.com/ubuntu noble-updates/main amd64 git-man all 1:2.43.0-1ubuntu7.2 [1 100 kB]
Réception de :8 http://ma.archive.ubuntu.com/ubuntu noble-updates/main amd64 git amd64 1:2.43.0-1ubuntu7.2 [3 679 kB]
Réception de :9 http://ma.archive.ubuntu.com/ubuntu noble/universe amd64 ubuntu-fan all 0.12.16 [35,2 kB]
```

### 2.3.2. Vérification de l'installation Docker

Après l'installation, nous avons vérifié que Docker fonctionne correctement en affichant la version installée avec la commande :

```
docker version
```

```
master3@master3-VirtualBox:~$ docker version
Client:
 Version:           26.1.3
 API version:       1.45
 Go version:        go1.22.2
 Git commit:        26.1.3-0ubuntu1~24.04.1
 Built:             Mon Oct 14 14:29:26 2024
 OS/Arch:           linux/amd64
 Context:           default
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.soc
k/v1.45/version": dial unix /var/run/docker.sock: connect: permission denied
```

```
master1@master1-VirtualBox:~$ sudo systemctl enable docker
```

Elle garantit que le service reste actif et disponible, même après un redémarrage du système

### 2.3.3. Activation du Démarrage Automatique de Docker

Pour assurer que le service Docker démarre automatiquement à chaque démarrage du système (afin que les conteneurs puissent être lancés sans intervention manuelle), nous avons activé le service Docker avec la commande :

**sudo systemctl enable docker**

```
master3@master3-VirtualBox:~$ sudo cat <<EOF | sudo tee /etc/docker/daemon.json
> { "exec-opts": ["native.cgroupdriver=systemd"],
> "log-driver": "json-file",
> "log-opts":
> { "max-size": "100m" },
> "storage-driver": "overlay2"
> }
> EOF
{ "exec-opts": ["native.cgroupdriver=systemd"],
"log-driver": "json-file",
"log-opts":
{ "max-size": "100m" },
"storage-driver": "overlay2"
}
```

Cette étape garantit que Docker reste actif et disponible, même après un redémarrage de la machine.

### 3.3.4. Configuration de Docker pour Kubernetes:

Pour optimiser la compatibilité entre Docker et Kubernetes, il est nécessaire de configurer Docker en modifiant son fichier de configuration /etc/docker/daemon.json. Ce fichier permet notamment de régler le système de

stockage, les options réseau, et d'autres paramètres essentiels au bon fonctionnement dans un cluster Kubernetes.

Cette étape, bien que non détaillée ici, consiste généralement à définir le driver de stockage (comme overlay2) et à configurer le système de journalisation.

```
master3@master3-VirtualBox:~$ sudo systemctl restart docker
master3@master3-VirtualBox:~$ sudo docker info | grep -i cgroup
 Cgroup Driver: systemd
 Cgroup Version: 2
  cgroupns
```

### 3.3.5. Désactivation de la Mémoire Swap pour Kubernetes

Kubernetes requiert que la mémoire swap soit désactivée pour garantir la stabilité et la performance du cluster. Nous avons donc désactivé la swap temporairement avec :

**sudo swapoff -a**

```
master3@master3-VirtualBox:~$ sudo swapoff -a
master3@master3-VirtualBox:~$ sudo nano /etc/fstab
```

Pour rendre cette désactivation permanente, nous avons édité le fichier /etc/fstab pour commenter la ligne correspondant à la partition swap :

**sudo nano /etc/fstab**

```
  GNU nano 7.2                                                  /etc/fstab *
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/2c15db6e-02b5-48d8-93f0-ae9754f0f394 / ext4 defaults 0 1
#/swap.img      none    swap    sw      0       0
```

En commentant cette ligne, la swap ne sera plus activée au démarrage suivant.

### 3.3.6. Activation des Modules Overlay et br_netfilter

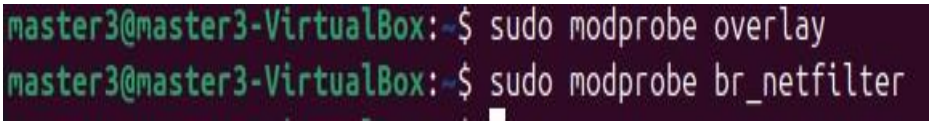
Docker et Kubernetes dépendent de certains modules noyau pour assurer la gestion du réseau et du stockage. Nous avons activé ces modules avec les commandes :

- Pour le module Overlay (gestion du système de fichiers) :

```
sudo modprobe overlay
```

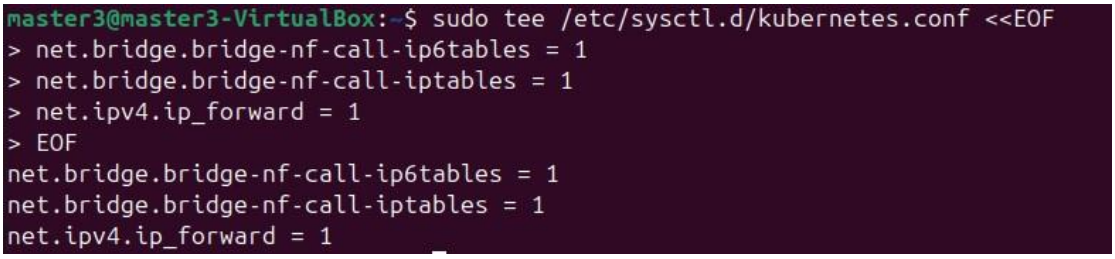
- Pour le module br_netfilter (gestion des règles de filtrage réseau) :

```
sudo modprobe br_netfilter
```




### 3.3.7. Configuration des Paramètres Réseau pour Kubernetes

Afin de garantir la bonne gestion du trafic réseau et la communication entre les pods, nous avons configuré des paramètres système spécifiques. Ces réglages ont été ajoutés dans un fichier dédié /etc/sysctl.d/kubernetes.conf avec les valeurs suivantes :




Ces paramètres permettent notamment au noyau Linux de router correctement les paquets réseau entre interfaces réseau pontées et d'activer le transfert IPv4.

### 3.3.8. Application des Paramètres Système

Après avoir sauvegardé les réglages, nous avons appliqué immédiatement ces modifications sans redémarrer la machine grâce à la commande :

**sudo sysctl --system**

```
master3@master3-VirtualBox:~$ sudo sysctl --system
* Applique /usr/lib/sysctl.d/10-apparmor.conf …
* Applique /etc/sysctl.d/10-bufferbloat.conf …
* Applique /etc/sysctl.d/10-console-messages.conf …
* Applique /etc/sysctl.d/10-ipv6-privacy.conf …
* Applique /etc/sysctl.d/10-kernel-hardening.conf …
* Applique /etc/sysctl.d/10-magic-sysrq.conf …
* Applique /etc/sysctl.d/10-map-count.conf …
* Applique /etc/sysctl.d/10-network-security.conf …
* Applique /etc/sysctl.d/10-ptrace.conf …
* Applique /etc/sysctl.d/10-zeropage.conf …
* Applique /usr/lib/sysctl.d/30-tracker.conf …
* Applique /usr/lib/sysctl.d/50-bubblewrap.conf …
* Applique /usr/lib/sysctl.d/50-pid-max.conf …
* Applique /usr/lib/sysctl.d/99-protect-links.conf …
* Applique /etc/sysctl.d/99-sysctl.conf …
* Applique /etc/sysctl.d/kubernetes.conf …
* Applique /etc/sysctl.conf …
kernel.apparmor_restrict_unprivileged_userns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
```

Cette commande recharge tous les fichiers de configuration sysctl et met à jour les paramètres actifs du noyau.

Ces étapes garantissent un environnement Docker parfaitement adapté pour l'intégration avec Kubernetes, assurant ainsi une infrastructure stable, performante et conforme aux prérequis de la plateforme d'orchestration.

## 2.4. Etape 4 : Configuration du Load Balancer pour le Cluster Kubernetes

Pour le **load balancer**, il n'est pas nécessaire d'installer ou de configurer les composants Kubernetes tels que **kubelet**, **kubeadm** ou **kubectl**, car ce serveur ne fait pas partie du cluster Kubernetes lui-même. Sa fonction principale est de répartir le trafic réseau entre les différents nœuds maîtres (control plane), garantissant ainsi la haute disponibilité du cluster.

### 3.4.1. Installation de Docker

Comme pour les nœuds master et worker, nous avons commencé par installer Docker sur la machine dédiée au load balancer afin d'assurer une gestion cohérente des services, même si Docker n'est pas strictement nécessaire pour HAProxy.

Nous avons également activé le démarrage automatique du service Docker.

### 3.4.2. Installation de HAProxy pour la haute disponibilité

Pour assurer une répartition efficace et fiable du trafic entrant vers les nœuds maîtres, nous avons choisi HAProxy, un logiciel libre reconnu pour ses performances en répartition de charge et en proxy TCP/HTTP.

L'installation s'est réalisée avec la commande suivante :

```
sudo apt install -y haproxy
```

```
load-balancer@load-balancer-VirtualBox:~$ sudo apt install -y haproxy
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Paquets suggérés :
  vim-haproxy haproxy-doc
Les NOUVEAUX paquets suivants seront installés :
  haproxy
0 mis à jour, 1 nouvellement installés, 0 à enlever et 111 non mis à jour.
Il est nécessaire de prendre 2 038 ko dans les archives.
Après cette opération, 4 635 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://ma.archive.ubuntu.com/ubuntu noble-updates/main amd64 haproxy amd64 2.8.5-1ubuntu3.3 [2 038 kB]
2 038 ko réceptionnés en 2s (968 ko/s)
Sélection du paquet haproxy précédemment désélectionné.
(Lecture de la base de données... 150858 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../haproxy_2.8.5-1ubuntu3.3_amd64.deb ...
Dépaquetage de haproxy (2.8.5-1ubuntu3.3) ...
Paramétrage de haproxy (2.8.5-1ubuntu3.3) ...
Created symlink /etc/systemd/system/multi-user.target.wants
/haproxy.service → /usr/lib/systemd/system/haproxy.service.
Traitement des actions différées (« triggers ») pour rsyslo
g (8.2312.0-3ubuntu9) ...
Traitement des actions différées (« triggers ») pour man-db
 (2.12.0-4build2) ...
```

### 3.4.3. Configuration Frontend et Backend de HAProxy

Nous avons ensuite configuré HAProxy pour définir les règles de répartition du trafic. Cela implique la configuration de deux sections principales dans le fichier /etc/haproxy/haproxy.cfg :

- **Frontend** : définit l'interface d'écoute pour les requêtes entrantes (adresse IP, port).
- **Backend** : liste les nœuds maîtres vers lesquels HAProxy va distribuer le trafic.

```
frontend kubernetes_frontend
        bind 192.168.60.10:6443
        mode tcp
        default_backend kubernetes_backend
backend kubernetes_backend
        mode tcp
        balance roundrobin
        server master1 192.168.60.11:6443
        server master2 192.168.60.12:6443
        server master3 192.168.60.13:6443
```

### 3.4.4. Redémarrage et vérification du service HAProxy

Après avoir sauvegardé le fichier de configuration, nous avons redémarré HAProxy pour appliquer les modifications :

**sudo systemctl restart haproxy**

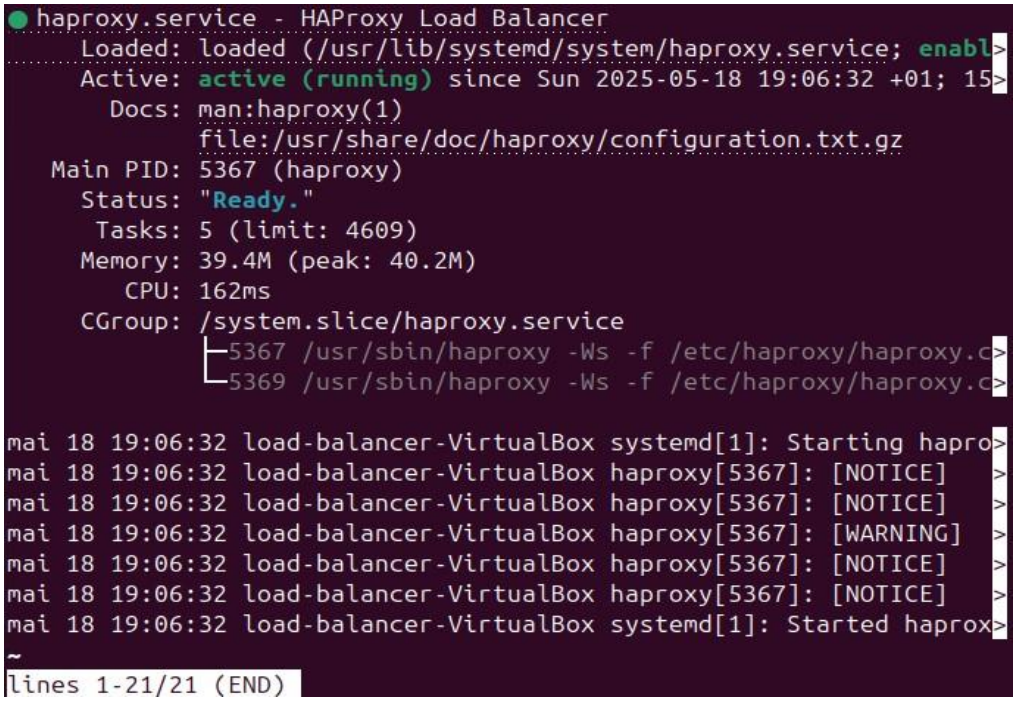
```
load-balancer@load-balancer-VirtualBox:~$ sudo nano /etc/haproxy/haproxy.cfg
load-balancer@load-balancer-VirtualBox:~$ sudo systemctl restart haproxy
```

Pour vérifier que HAProxy fonctionne correctement, nous avons utilisé :

**systemctl status haproxy**

Cette commande confirme que le service est actif et qu'il n'y a pas d'erreurs dans la configuration.

Cette configuration du load balancer garantit une haute disponibilité du plan de contrôle Kubernetes en distribuant efficacement le trafic entre les nœuds maîtres, ce qui est essentiel pour la résilience et la fiabilité du cluster.



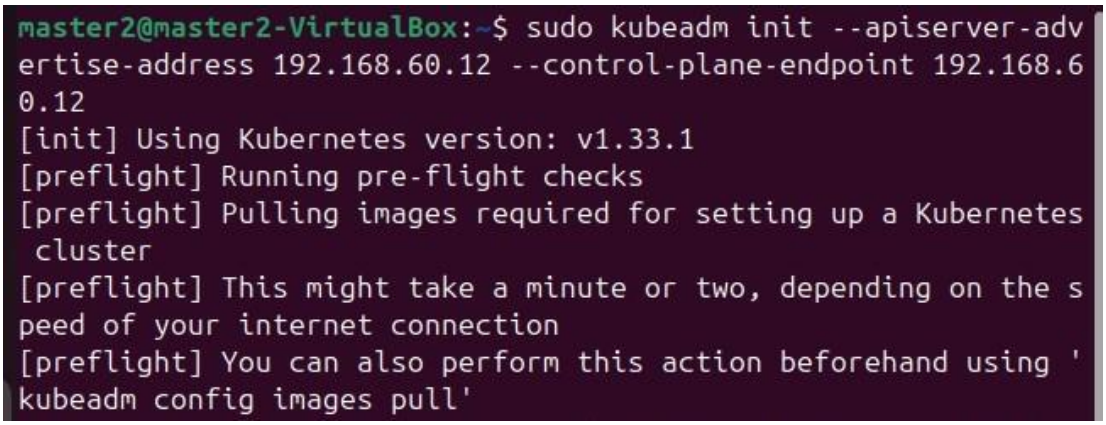

## 2.5. Etape 5: Initialisation des premiers nœuds du cluster

Pour démarrer la mise en place de notre cluster Kubernetes, nous avons d'abord initialisé un nœud master, puis ajouté trois nœuds workers.

### 2.5.1. Initialisation du nœud master avec kubeadm

La première étape a consisté à initialiser le nœud master, qui sera le centre de contrôle du cluster. Nous avons exécuté la commande suivante :

**sudo kubeadm init --apiserver-advertise-address 192.168.60.11 --control-plane-endpoint 192.168.60.11**



- L'option **--apiserver-advertise-address** spécifie l'adresse IP utilisée par l'API server pour communiquer avec les autres composants du cluster.
- **--control-plane-endpoint** désigne l'adresse utilisée pour accéder au plan de contrôle, ici identique à l'adresse du master.

À l'issue de cette commande, kubeadm génère un token sécurisé ainsi qu'une commande complète permettant d'ajouter d'autres nœuds workers au cluster.

Pour récupérer cette commande d'ajout, nous avons exécuté :

```
kubeadm token create --print-join-command
```

```
master2@master2-VirtualBox:~$ kubeadm token create --print-join
-command
kubeadm join 192.168.60.12:6443 --token z5yvj9.pfjckm1mbu4uzczz
 --discovery-token-ca-cert-hash sha256:e56191a31f9e4318f706194e
ff50440a6896a9a4ea964eaf9abdf381718a40b1
```

### 2.5.2. Configuration de kubectl pour Accéder au Cluster

Pour pouvoir interagir avec le cluster depuis notre session utilisateur, il faut configurer l'outil kubectl :

Nous avons créé un répertoire dédié à la configuration :

```
mkdir -p $HOME/.kube
```

Puis copié le fichier de configuration généré par kubeadm dans ce répertoire :

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

Enfin, nous avons modifié les permissions afin que l'utilisateur courant puisse accéder à ce fichier :

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Cette configuration permet à kubectl d'utiliser les certificats et accès nécessaires pour communiquer avec le cluster Kubernetes.

### 2.5.3. Vérification de l'État des Nœuds

Nous avons contrôlé que le nœud master était bien initialisé et opérationnel avec la commande :

**kubectl get nodes**

Le nœud doit apparaître avec le statut Ready, indiquant qu'il est prêt à gérer les pods et ressources.

```
master2@master2-VirtualBox:~$ kubectl get nodes
NAME                STATUS     ROLES           AGE     VERSION
master2-virtualbox   Ready      control-plane   119m    v1.33.0
worker1-virtualbox   NotReady   <none>          9m19s   v1.33.0
worker2-virtualbox   NotReady   <none>          4m48s   v1.33.1
worker3-virtualbox   NotReady   <none>          6s      v1.33.1
```

### 2.5.4. Installation du réseau Kubernetes

Pour permettre la communication entre les différents pods déployés dans le cluster, il est indispensable d'installer un plugin réseau. Nous avons choisi **Flannel**, qui est simple et adapté aux clusters locaux.

L'installation s'effectue via la commande :

**kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml**

```
master2@master2-VirtualBox:~$ kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-fl
annel.yml
namespace/kube-flannel unchanged
clusterrole.rbac.authorization.k8s.io/flannel unchanged
clusterrolebinding.rbac.authorization.k8s.io/flannel unchanged
serviceaccount/flannel unchanged
configmap/kube-flannel-cfg unchanged
```

Après cette étape, le réseau est configuré, et les nœuds workers pourront rejoindre le cluster tout en communiquant efficacement avec les autres composants.

### 2.5.5. Vérification finale des nœuds

Après avoir rejoint le cluster et configuré le réseau, nous avons revérifié l'état des nœuds avec kubectl get nodes. Leur statut est passé de NotReady à Ready, signe que la configuration réseau est fonctionnelle et que les nœuds sont pleinement opérationnels.

```
master2@master2-VirtualBox:~$ kubectl get nodes
NAME                 STATUS   ROLES           AGE     VERSION
master2-virtualbox   Ready    control-plane   2d16h   v1.33.0
worker1-virtualbox   Ready    <none>          2d14h   v1.33.0
worker2-virtualbox   Ready    <none>          2d14h   v1.33.1
worker3-virtualbox   Ready    <none>          2d14h   v1.33.1
```

Cette procédure a permis de mettre en place un cluster Kubernetes local fonctionnel, prêt à accueillir et orchestrer des applications conteneurisées.

## 2.6. Etape 6: Déploiement d'une application simple(Nginx)

Pour valider le bon fonctionnement de notre cluster Kubernetes, nous avons déployé une application simple basée sur Nginx, un serveur web populaire. Voici les différentes étapes suivies :

### 2.6.1. Création du fichier YAML de déploiement

Nous avons créé un fichier nommé **nginx-deployment.yaml** qui décrit le déploiement de l'application Nginx. Pour cela, nous avons ouvert un éditeur de texte :

```
Sudo nano nginx-deployment.yaml
```

Ce fichier contient la configuration nécessaire pour créer un déploiement Kubernetes, spécifiant le nombre de réplicas, l'image Docker utilisée, et les ports exposés.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
```

```yaml
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30001
```

### 2.6.2. Application du fichier YAML

Nous avons ensuite appliqué cette configuration au cluster avec la commande suivante:

**kubectl apply -f nginx-deployment.yaml**

```
master2@master2-VirtualBox:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
service/nginx-service created
```

Cette commande crée les ressources Kubernetes définies dans le fichier : déploiement, pods, et éventuellement services.

### 2.6.3. Vérification des pods

Pour vérifier que les pods ont bien été créés et sont en cours d'exécution, nous avons exécuté:

**kubectl get pods**

```
master2@master2-VirtualBox:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS         AGE
nginx-deployment-96b9d695-7mgrw     1/1     Running   8 (47s ago)      5h52m
nginx-deployment-96b9d695-f85sh     1/1     Running   13 (3m35s ago)   5h52m
nginx-deployment-96b9d695-k4qbt     1/1     Running   3 (66s ago)      5h52m
```

Le statut des pods doit être **Running**.

### 2.6.4. Vérification du déploiement

Pour lister les déploiements actifs dans le cluster, nous avons utilisé :

**Kubectl get deployment**

```
master2@master2-VirtualBox:~$ kubectl get deployment
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           5h54m
```

Le déploiement **nginx-deployment** doit apparaître avec le nombre de réplicas désiré.

### 2.6.5. Vérification des services

Nous avons aussi vérifié que le service permettant d'exposer l'application a été créé :

**kubectl get services**

```
master2@master2-VirtualBox:~$ kubectl get service
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes      ClusterIP   10.96.0.1       <none>        443/TCP        28d
nginx-service   NodePort    10.105.143.56   <none>        80:30001/TCP   2d
```

Cette commande liste les services et leurs ports d'écoute, notamment le port externe configuré.

### 2.6.6. Vérification de l'Accessibilité de l'Application

Pour s'assurer que Nginx est accessible depuis l'extérieur du cluster, nous avons testé l'accès via un navigateur ou curl, en utilisant l'adresse IP des nœuds workers et le port exposé par le service (ici 30001) :

**http://192.168.60.21:30001**

**http://192.168.60.22:30001**

**http://192.168.60.23:30001**

La page d'accueil Nginx devrait s'afficher, confirmant l'accessibilité.

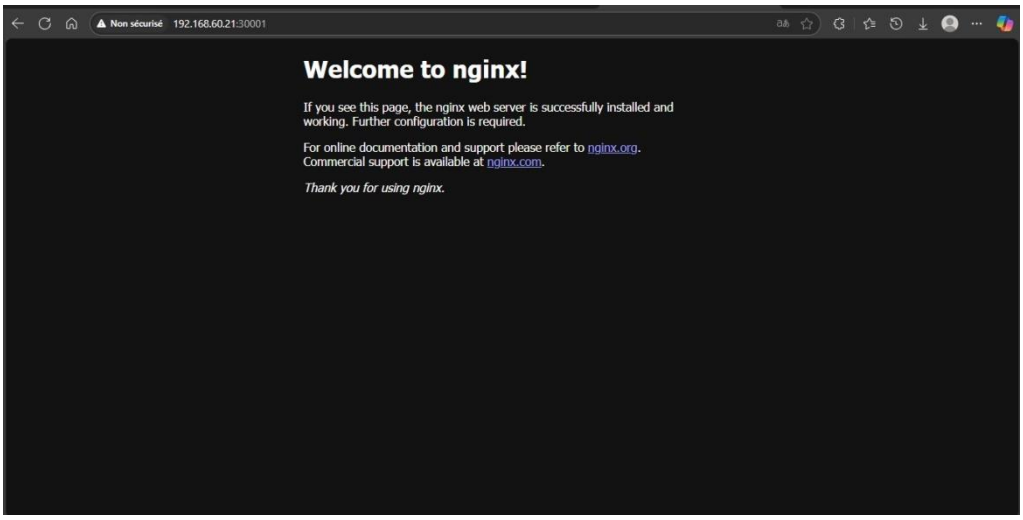
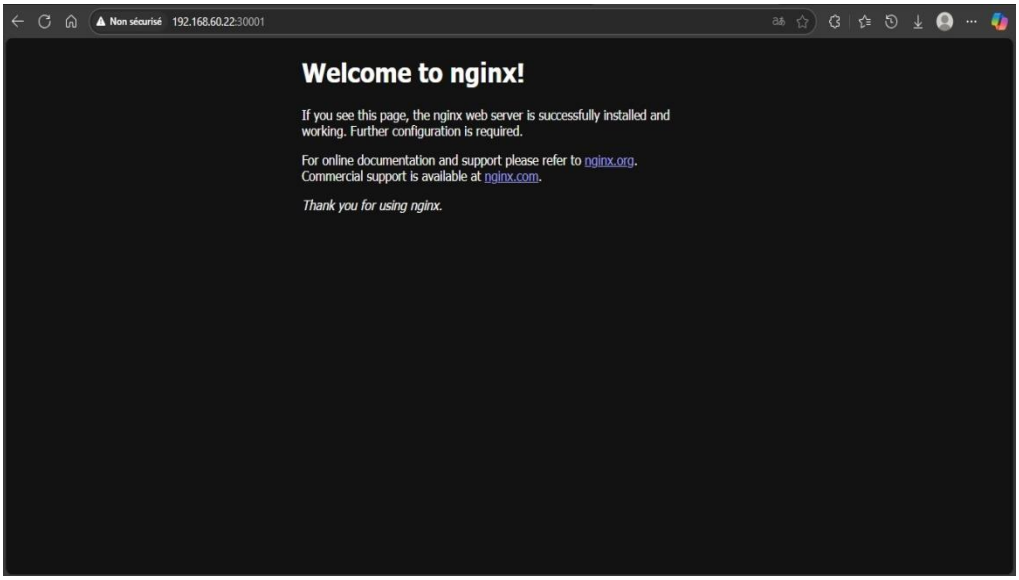
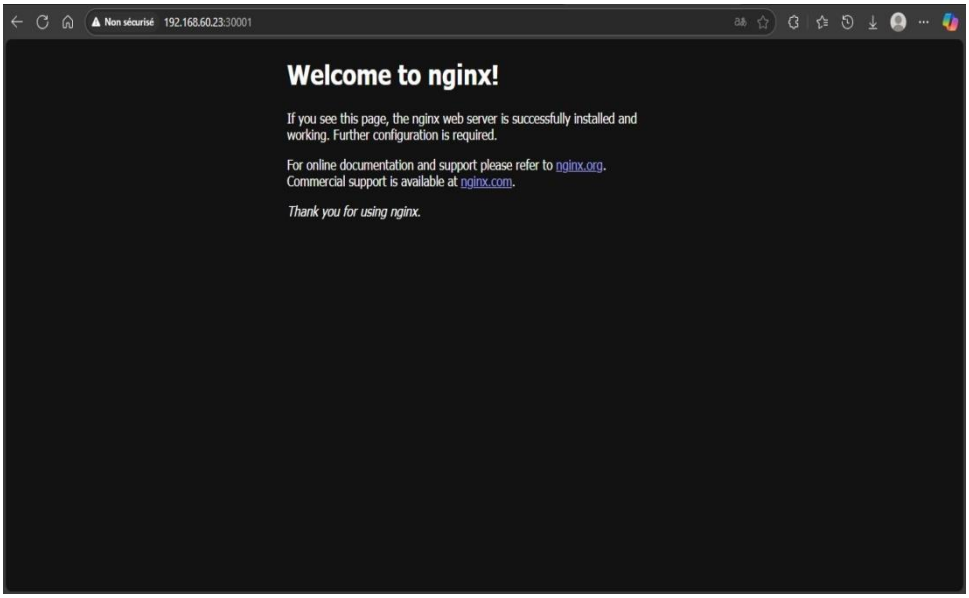



Figure 40: La page d'accueil Nginx




### 3.6.7. Simulation d'une Panne de Nœud Worker

Pour tester la résilience du cluster, nous avons simulé une panne sur un nœud worker :

- D'abord, nous avons identifié sur quel nœud chaque pod était déployé avec:

```
kubectl get pods -o wide
```

- Puis, nous avons éteint le nœud worker1 :

```
sudo shutdown now
```

On vérifie l'état des noeuds, on remarque que le premier noeud est marqué comme NotReady:

```
master2@master2-VirtualBox:~$ kubectl get nodes
NAME                STATUS     ROLES           AGE     VERSION
master2-virtualbox  Ready      control-plane   119m    v1.33.0
worker1-virtualbox  NotReady   <none>          9m19s   v1.33.0
```

```
worker2-virtualbox   Ready    <none>          2d14h   v1.33.1
worker3-virtualbox   Ready    <none>          2d14h   v1.33.1
```

- Enfin, nous avons observé que les pods qui étaient sur ce nœud ont été automatiquement redéployés sur les autres nœuds restants. Le cluster a ainsi assuré la continuité de service.

```
master2@master2-VirtualBox:~$ kubectl get pods -o wide
NAME                                READY   STATUS             RESTARTS         AGE     IP             NODE                NOMINATED NODE   READINESS GATES
nginx-deployment-96b9d695-7mgrw     1/1     Running            14 (2m6s ago)    6h15m   10.244.4.24    worker3-virtualbox  <none>           <none>
nginx-deployment-96b9d695-f85sh     0/1     CrashLoopBackOff   15 (3m58s ago)   6h15m   10.244.2.25    worker1-virtualbox  <none>           <none>
nginx-deployment-96b9d695-k4qbt     1/1     Running            8 (7m52s ago)    6h15m   10.244.3.9     worker2-virtualbox  <none>           <none>
```

- Nous avons confirmé que l'application Nginx restait accessible via les nœuds actifs.

Cette expérience confirme que notre cluster Kubernetes assure la haute disponibilité et la résilience nécessaires pour maintenir les applications en fonctionnement, même en cas de défaillance d'un nœud.

## 2.7. Conclusion

La mise en place du cluster Kubernetes en environnement local nous a permis d'acquérir une compréhension pratique des différentes étapes nécessaires à la création, la configuration et la gestion d'un cluster. Grâce à l'installation et la configuration des nœuds Master et Worker, ainsi qu'au déploiement du plugin réseau Flannel, nous avons pu constituer une infrastructure stable et fonctionnelle.

Le test avec l'application Nginx a validé la capacité du cluster à déployer des pods, à gérer la communication réseau entre eux, et à assurer la haute disponibilité grâce à la redondance des nœuds. La simulation d'une panne sur un nœud Worker a démontré la robustesse et la résilience du système, capable de redistribuer automatiquement les charges de travail sans interruption de service.

Cette phase pratique illustre parfaitement les avantages de Kubernetes pour l'orchestration des conteneurs dans des environnements distribués, et pose les bases pour la réalisation de projets plus complexes et évolutifs dans le futur.

# Conclusion générale

La mise en place d'un cluster Kubernetes illustre une méthode moderne et efficace pour gérer des applications sous forme de conteneurs, c'est-à-dire des petits modules logiciels autonomes. Kubernetes simplifie l'administration en automatisant des tâches importantes comme le déploiement des applications, la montée en charge automatique lorsque le nombre d'utilisateurs augmente, et la gestion des pannes en redémarrant automatiquement les services défaillants.

Au cours de ce projet, nous avons découvert comment organiser un cluster, comprendre le rôle des nœuds maîtres et des nœuds workers, et configurer un réseau permettant la communication entre les différentes parties. Nous avons aussi appris à déployer une application simple pour tester le fonctionnement du cluster, à surveiller l'état des ressources, et à gérer des situations concrètes comme la panne d'un nœud.

Cette expérience pratique permet de mieux saisir les avantages d'une telle technologie dans le monde professionnel, où il est essentiel d'assurer une haute disponibilité des services tout en optimisant l'utilisation des ressources informatiques. Elle nous donne les bases pour intervenir sur des infrastructures plus complexes, souvent utilisées dans les environnements DevOps et cloud natifs, où la rapidité, la flexibilité et la résilience des systèmes sont cruciales.

En résumé, ce projet nous a permis de passer de la théorie à la pratique, de comprendre les enjeux de l'orchestration des conteneurs, et d'acquérir des compétences techniques clés qui sont aujourd'hui très recherchées dans le secteur informatique.

# Références


- https://kinsta.com/fr/base-de-connaissances/qu-est-ce-que-docker/
- https://geekflare.com/fr/docker-architecture/
- https://kerneltalks.com/virtualization/beginners-guide-to-docker-image/
- https://medium.com/@mojabi.rafi/create-a-kubernetes-cluster-using-virtualbox-and-without-vagrant-90a14d791617
- https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/
- https://share.google/NjzHVvJML0cGtRxhN
- https://share.google/UA5TKhyLrTbaHIvhM
- https://share.google/xfXRyBZSlCMRkrgw8
- https://share.google/3DUqWI4JbslpZlAKS

**Résumé :**

Ce projet de fin d'études est le fruit de notre parcours académique au sein de la Faculté des Sciences Ain Chock, visant à obtenir le diplôme de licence en sciences mathématiques et informatique. Il se focalise sur la mise en place d'un cluster Kubernetes robuste, dans le but de moderniser les infrastructures informatiques et de répondre aux exigences actuelles en matière de déploiement, de gestion et de montée en charge des applications. Nous mettons en œuvre nos compétences techniques en utilisant des outils tels que Docker, Kubernetes, VirtualBox et Ubuntu pour concevoir une infrastructure flexible, performante et adaptée aux pratiques DevOps.

L'analyse initiale des technologies existantes et des besoins spécifiques du projet nous a permis d'identifier les objectifs clés : faciliter le déploiement et la gestion des applications, et garantir une infrastructure fiable et scalable. En adoptant une approche méthodologique rigoureuse, nous avons modélisé l'architecture théorique du cluster Kubernetes, en mettant l'accent sur des concepts tels que la haute disponibilité, la répartition de charge et l'optimisation des ressources. Cette phase préparatoire a été essentielle pour assurer une transition fluide vers la mise en œuvre pratique.

La réalisation technique du projet repose sur un environnement simulé avec VirtualBox, permettant de créer un cluster composé d'un nœud maître (master node) et de trois nœuds workers. Nous détaillons étape par étape la configuration réseau, l'installation des machines virtuelles et le déploiement des outils nécessaires, validant ainsi les notions théoriques par une application concrète. L'utilisation de Docker pour la conteneurisation et de Kubernetes pour l'orchestration des conteneurs garantit une infrastructure cohérente et automatisée.

La planification rigoureuse du projet, illustrée par des étapes claires et méthodiques, nous a guidés tout au long des phases de conception, de développement et de déploiement. En résumé, ce projet répond aux besoins croissants de modernisation des infrastructures informatiques en proposant une solution flexible, performante, soutenue par des outils